



400-650-7088

***Tong*Web** V7.0

应用管理指南

版本：7049_M9A01

2025 年 09 月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech**标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

手册版本	适用于产品版本	更新内容
7049_M9A01	7.0.4.9_M9	无
7049_M8A01	7.0.4.9_M8	无
7049_M7A01	7.0.4.9_M7	无
7049_M6A01	7.0.4.9_M6	无
7049_M5A01	7.0.4.9_M5	修改如下章节： ● 3.2.2.1 自动部署设置，新增“部署超时时间”，自动部署超过部署超时时间，系统将中断部署流程。 ● 4.3.1 deploy，新增“--deployTimeout”。 ● 4.3.7 update-auto-deploy-config，新增“--deployTimeout”。
7049_M4A01	7.0.4.9_M4	无
7049_M3A01	7.0.4.9_M3	无
7049_M2A01	7.0.4.9_M2	无
7049_M1A01	7.0.4.9_M1	无
7049A01	7.0.4.9	无
7048A01	7.0.4.8	无
7047A01	7.0.4.7	● 新增如下章节： 附录 A.2.1 新增tongweb-web.xml属性配置顺序说明。 ● 修改如下章节： ■ “3.2.1 控制台部署”章节，新增session管理相关描述。 ■ “3.2.1 控制台部署”章节，新增部署超时配置。 ■ “3.2.1 控制台部署”章节，新增类热加载的配置。
7046A01	7.0.4.6	● 新增如下章节： ■ 4.1.8 备份应用 ■ 4.3.4 show-backup-appdir ■ 4.3.5 update-backup-appdir ■ 4.3.6 backup-app ● 修改如下章节： ■ “8.3类加载推荐策略”，添加7.同一目录下jar文件加载顺序的配置。 ■ “3.2.2.1 自动部署设置”更新“图3.2-12：开启自动部署”，增加“自动部署虚拟主机”配置选项。 ■ “3.2.3.1 热部署设置”更新“图3.2-13：开启热部署”。
7045A01	7.0.4.5	● “3.2.1控制台部署”及“7.1文件集”中增加了服务器显示目录的说明及截图。 ● 将\${TW_HOME}改为\${TongWeb_HOME} ● 更新“图3.2-4在服务器上选择部署文件”及添加服务器部署

手册版本	适用于产品版本	更新内容
		应用说明 ● 在“8.4类加载参数”章节添加公有库、私有库类加载说明 ● 更新“图7.1-3选择服务器上的文件集”及添加服务器选择文件可选目录说明 ● 在“8.3类加载推荐策略”中添加Web、EAR应用的配置说明
7044A01	7.0.4.4	● 删除3.2.4命令行部署、4.3命令行管理中远程部署的内容。 ● 更新表2.1-1及2.5中其他应用的扩展名为.car ● 更新“图3.2-4在服务器上选择部署文件”及添加说明
7043A01	7.0.4.3	无
7042A01	7.0.4.2	无
7041A01	7.0.4.1	1. 统一使用TongWeb用户使用手册模板。 2. 整合原《TongWeb7用户手册》第三章“应用管理”，附录1“应用配置说明”。

前言

本文档是TongWeb产品的用户使用手册之一，详细说明TongWeb支持部署及管理的应用类型，提供四种部署方式的详细步骤，其他管理操作的方法，以及应用配置文件的具体属性说明。

适合的对象

本手册主要适用于生产环境中的系统管理员，部分内容同样适用于应用开发人员和应用部署人员。

本手册假定您已经具备如下技能：

1. 基本系统管理任务；
2. 安装软件；
3. 使用Web浏览器；
4. 启动数据库服务器；
5. 在终端窗口中发布命令。

用户使用手册集

TongWeb为您提供的用户使用手册集包含的文档有：

- 000_TongWeb_V7.0用户手册导读：各版本手册集的目录及手册版本说明。
- 001_TongWeb_V7.0快速使用手册：提供各版本客户快速安装，应用部署，及常用配置的说明。
- 002_TongWeb_V7.0产品简介及安装指南：提供各版本各平台上的安装启动的步骤。
- 003_TongWeb_V7.0服务配置指南：提供Web容器、EJB容器、启动参数配置及各项基础服务的配置说明。
- 004_TongWeb_V7.0应用管理指南：提供管理控制台对应用进行部署和管理操作的说明。
- 005_TongWeb_V7.0工具使用指南：提供Eclipse、IDEA、Patch、TongAPM、appclient、pinpoint、web应用迁移等工具的使用说明。
- 006_TongWeb_V7.0Commandstool使用指南：提供命令行Commandstool的使用说明，基本参数及所有命令示例。
- 007_TongWeb_V7.0与Tivoli集成手册：对JSON整体结构及各监视项的说明。
- 008_TongWeb_V7.0二次开发接口：TongWeb的JMX、REST、SNMP接口说明。
- 009_TongWeb_V7.0应用开发手册：提供在TongWeb上开发应用的各类规范说明。
- 010_TongWeb_V7.0等级保护指南：提供等级保护功能开启后的使用说明。
- 100_TongWeb_V7.0集群管理指南：详细介绍集群的配置和管理。
- 200_TongWeb_V7.0用户使用手册-轻量级版：提供轻量级版本客户详细的服务配置及应用管理的说明。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

网址：www.tongtech.com

电话：400-650-7088

邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

1. 您的姓名
2. 您的公司信息
3. 您的联系方式
4. 操作系统及其版本
5. 产品版本号
6. 日志等错误的详细信息

目 录

第1章 TongWeb应用管理概述	1
第2章 应用类型	2
2.1 Web应用	2
2.2 EJB应用	3
2.3 Connector应用	5
2.4 EAR应用	5
2.5 其他应用	6
第3章 应用部署	8
3.1 部署源的两类型	8
3.1.1 文件部署	8
3.1.2 目录部署	8
3.2 应用部署的四种方式	8
3.2.1 控制台部署	8
3.2.2 自动部署	16
3.2.3 热部署	18
3.2.4 命令行部署	19
第4章 应用管理	21
4.1 使用管理控制台	21
4.1.1 查看应用	21
4.1.2 编辑应用	21
4.1.3 访问应用	23
4.1.4 停止应用	24
4.1.5 启动应用	24
4.1.6 解部署应用	25
4.1.7 重部署应用	25
4.1.8 备份应用	27
4.1.9 更新应用	27
4.2 自动扫描	29
4.2.1 自动解部署	29
4.2.2 自动重部署	30
4.3 命令行管理	30
4.3.1 deploy	30
4.3.2 undeploy	31
4.3.3 redeploy	32

4.3.4 show-backup-appdir.....	32
4.3.5 update-backup-appdir	32
4.3.6 backup-app.....	33
4.3.7 update-auto-deploy-config.....	33
4.3.8 list-apps.....	34
4.4 接口管理	34
第5章 Connector应用	35
5.1 Connector应用部署.....	35
5.2 Connector应用查看与编辑.....	35
5.3 Connector应用重部署.....	35
5.4 Connector应用解部署.....	35
第6章 虚拟目录	36
第7章 资源	37
7.1 文件集	37
7.2 共享库	39
第8章 类加载	41
8.1 类加载机制	41
8.2 类加载模式	42
8.2.1 父优先	42
8.2.2 子优先	43
8.3 类加载推荐策略	43
8.4 类加载参数	46
附录A 应用配置文件	47
A.1 default-web.xml	47
A.2 tongweb-web.xml	48
A.2.1 属性配置顺序	48
A.2.2 属性配置说明	49
A.3 ejb-jar.xml.....	53
A.4 tongweb-ejb-jar.xml.....	58
A.5 persistence.xml	59
附录B 术语及缩略语	60

第1章 TongWeb应用管理概述

企业级应用以服务器为中心，用户通过网络使用应用所提供的服务。一般来说，企业级应用应当具有的特征包括：并发支持、事务支持、交互支持、集群支持、Web支持、安全支持等。这些特征需要基础运行环境的支撑，TongWeb作为一款符合Java EE标准体系的应用服务器，可以很好地为企业级应用提供运行环境，从而最大化的降低应用的开发难度。

应用管理是TongWeb管理控制台上提供的最重要的功能模块。关于TongWeb应用管理，有以下几个“5”两个“4”和一个“2”。

一个“5”：TongWeb支持Web应用文件（.war）、EJB 应用文件（.jar）、Connector应用文件（.rar）、EAR应用文件（.ear）以及其他应用文件（.car），共计5种Java EE应用文件。

第一个“4”：TongWeb支持管理控制台部署、自动部署、热部署、命令行部署，共计4种部署方式。

第二个“4”：TongWeb支持在管理控制台中管理应用、自动扫描（自动解部署和自动重部署）、命令行管理和接口管理，共计4种应用管理方式。

一个“2”：TongWeb支持文件部署和目录部署2种部署源类型。

第2章 应用类型

TongWeb 支持五种 Java EE 应用文件的类型，包括 Web 应用、EJB 应用、Connector 应用、EAR 应用、其他。

表 2.1-1：支持的五种应用类型

类型	扩展名	用途和构成
Web 应用	.war	包含 Servlet 和 JSP 等 Web 组件，EJB 组件以及静态 HTML 页面、Jar 文件、标记库等
EJB 应用	.jar	包含 EJB 实现以及 EJB 实现所需的类
Connect 应用	.rar	包含连接器（资源适配器）的实现类
EAR 应用	.ear	包含上述三种应用类型
其它应用	.car	非标准格式的其他应用，选择此类应用文件后，只能通过后台获取其应用类型

2.1 Web应用

Web 应用指的是可以通过浏览器访问的应用。Web 应用（WAR 应用）的扩展名为**.war**，由动态资源（如 JSP、Servlet、JSP Tag 库等）和静态资源文件（如 HTML 页面和图片文件等）组成，Web 应用的具体结构如下图所示。

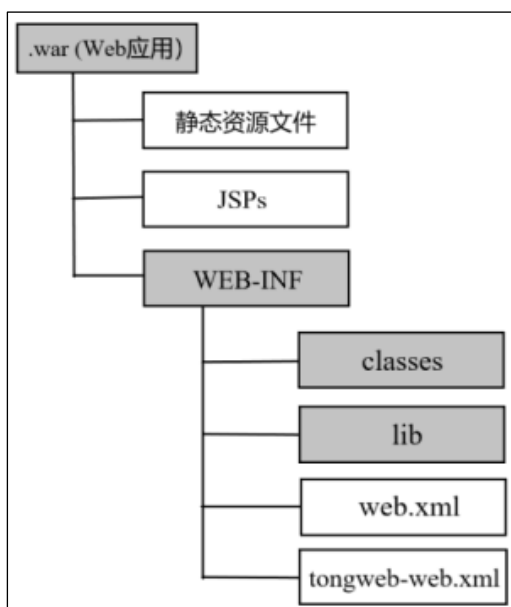


图 2.1-1：Web应用的结构

WEB-INF 用来存储服务端配置文件信息和在服务端运行的类文件。WEB-INF 存储的文件不允许客户端直接访问，WEB-INF 存储的各文件介绍如下。

- **classes**: 该目录下存放的是Web应用所需的类和资源，例如Servlet、EJB、工具类以及JavaBeans组件等。
- **lib**: 该目录存放的是Web应用所需的类包。
- **web.xml**: Java EE标准的部署描述文件。
- **tongweb-web.xml**: TongWeb自定义的部署描述文件。具体元素属性见附录A.2。

在 Java EE7 规范中，由于应用的配置可以通过原注释的方式写到类中，因此可以允许没有 web.xml 文件和 tongweb-web.xml 文件。

下图所示的 test.war 就是一个符合 Java EE7 规范的典型 Web 应用，其结构满足 Web 应用由动态资源（如 JSP、Servlet、JSP Tag 库等）和静态资源文件（如 HTML 页面和图片文件等）组成。

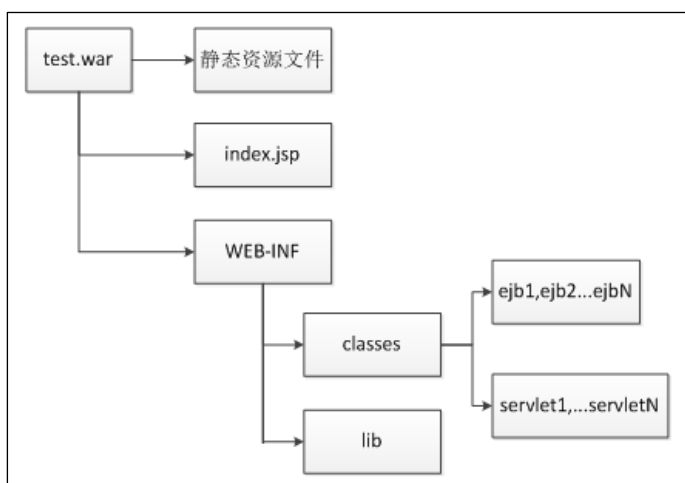


图 2.1-2: Web应用test.war

在 test.war 中，classes 既可以是传统的小服务程序（Servlet），也可以是 EJB 应用。Servlet（Server Applet）是 Java Servlet 的简称，称为小服务程序或服务连接器，是用 Java 编写的服务器端程序，其具有独立于平台和协议的特性，主要功能在于生成动态 Web 内容和数据，实现交互式地浏览效果。因为传统部署描述文件 web.xml 已经省略掉，应用的配置需要通过 Java EE 7 的注释定义在 servlet 类中。

2.2 EJB应用

EJB (Enterprise Java Beans) 是基于分布式事务处理的企业级应用的组件。EJB 是用于开发和部署多层结构、分布式、面向对象的 Java 应用系统的跨平台的构件体系结构。

EJB 应用可以部署在符合 Java EE 标准规范的应用服务器上。应用服务器及 EJB 容器提供了对象事务处理、日志记录、负载均衡、持久性机制、异常处理等系统级的服务。因此，开发者只需重点关注应用的业务逻辑，从而可以大幅简化大型企业级应用的开发工作。

一个EJB应用应包含EJB实现以及EJB实现所需的类，EJB 应用的具体结构如下图所示。

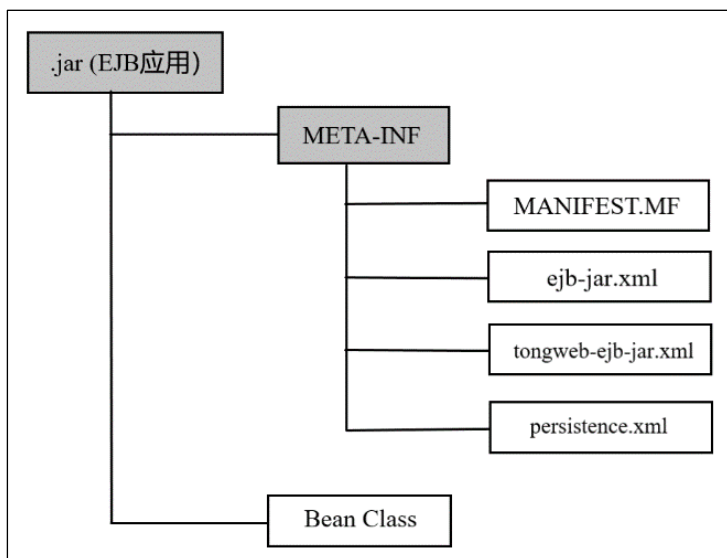


图 2.2-1: EJB应用的结构

META-INF 文件夹相当于一个信息包，在 META-INF 中包含的目录和目录中的文件获得了 Java 平台的认可与解释，可以用来配置应用、扩展程序、类加载器和服务。该文件夹和其中的 MANIFEST.MF 文件，是在 JAR 打包时自动生成的。META-INF 可以包含如下配置文件。

- **ejb-jar.xml**: Java EE标准部署描述文件。具体元素属性见附录A.3。
- **tongweb-ejb-jar.xml**: TongWeb自定义的部署描述文件。具体元素属性见附录A.4。
- **persistence.xml**: Entity使用JPA时，所需的持久化配置文件。具体元素属性见附录A.5。

EJB的JAR文件的配置定义，完全可以通过原注释的方式定义在EJB的类中。

下图所示的 test.jar 是一个符合 EJB3.1 以上规范的 EJB 应用的 JAR 包，其 ejb-jar.xml 已经不再是必选的部分。

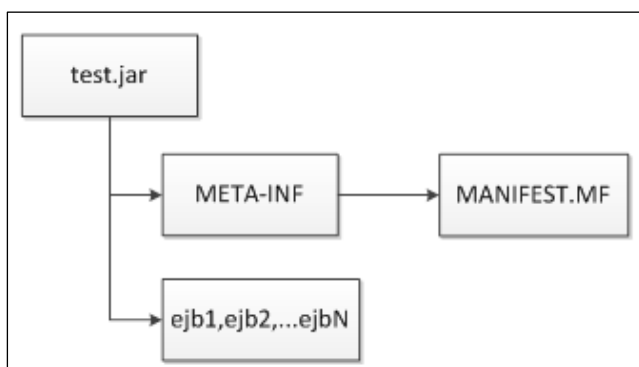


图 2.2-2: EJB应用test.jar

2.3 Connector应用

Connector 应用（连接器应用）是一种资源适配器。在扩展名为**.rar**的文件里，包含了资源适配器的实现类，它允许 Java EE 应用组件访问企业信息信息系统的底层资源管理器，并与之交互。Connector 应用主要由符合 JCA 规范的 Connector 应用类(JAR 包形式)和符合 Java EE 标准的部署描述文件 **ra.xml** 组成，具体结构如下图所示。

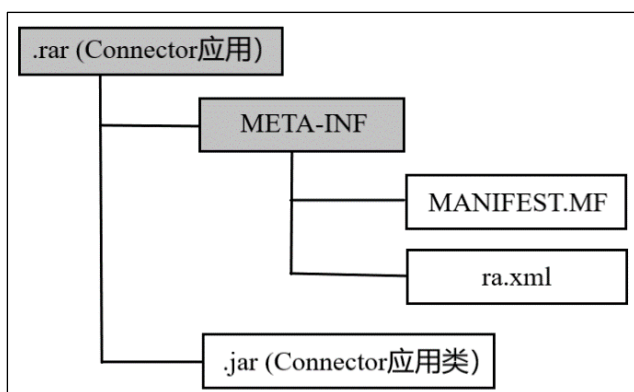


图 2.3-1: Connector应用的结构

2.4 EAR应用

EAR 应用是企业级应用的项目集合，它将应用所包括的子项目的多个 jar 文件和 war 文件打包在一个 ear 文件中，以便在应用服务器中整体部署。EAR 应用可包含 Web 应用（.war）、EJB 应用（.jar）和公用包。一个 EAR 应用可以包含一个或多个 Java EE 应用或者组件，EAR 应用的具体结构如下图所示。

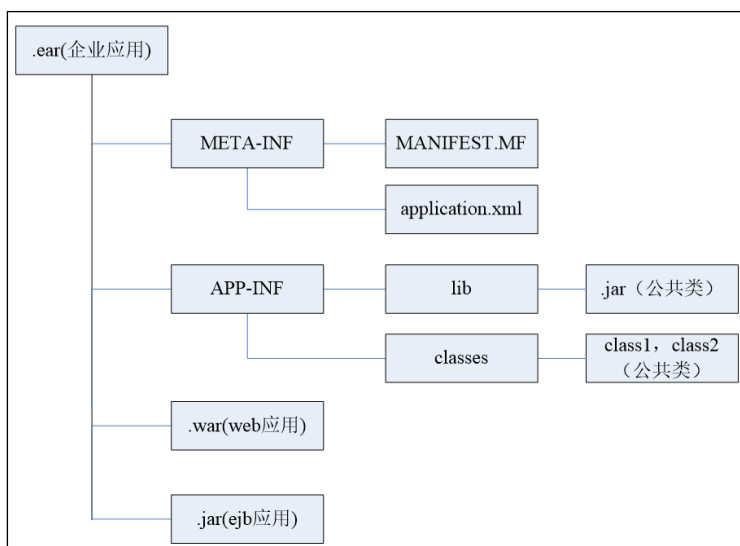


图 2.4-1: EAR 应用的结构

其中 **application.xml** 是 Java EE 标准的部署描述文件，**APP-INF** 是 EAR 应用的公共类。

下图所示的 test.ear 是一个典型的 EAR 应用。

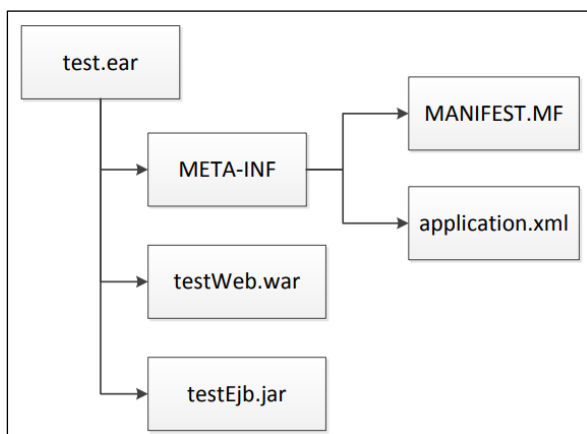


图 2.4-2: EAR应用test. ear

2.5 其他应用

TongWeb 还支持 .car 扩展名的应用文件(指标准格式 jar、war、ear、rar 之外的扩展名),CAR 是 Component Archive 的缩写。

J2EE 的 WEB 应用组件有一个严重的问题，问题来源于 J2EE 平台的 class loading 的模式：

- 每个WEB应用组件都独占一个classloader，这两个classloader所装载的类是无法共享的。换句话说，同一个类在两个WEB应用中可能有两个副本，各不相干。这将导致下列问题：
- 导致了大量内存的浪费。特别是当EAR中包含许多个WAR文件的时候。

- 导致另人费解的ClassCastException异常。这是因为不同classloader所装载的类的实例是不能互换的。而不同的WAR文件却是共享同一个EAR class loader。因此在某些情况下（例如cache），一个WAR下的对象可能会跑到另一个WAR的程序中，从而导致错误。
- 导致类的初始化问题。如果有一个被两个WAR所共享的类（位于EAR class loader），需要初始化。那么首先执行的WAR会抢先初始化该类，并用当前WAR的参数来初始化它。当第二个WAR被启动时，这个类的初始化状态有可能会发生错误。例如，在同一个EAR中同时运行两个使用Jakarta Turbine框架的WEB应用，就会发生这样的问题。org.apache.turbine.Turbine类会被第一个WEB应用初始化，而第二个WEB应用将失败。

Webx Framework 通过引进了一种新的组件：Webx Component（文件名后缀为 car），很好地解决了这个问题。事实上，CAR 文件的内部结构和 WAR 差不多，只是少了下列文件或目录：

- WEB-INF/web.xml描述文件。
- WEB-INF/lib目录。

与 WAR 组件一样，CAR 可以单独开发和调试，但在布署阶段，多个 CAR 被合并成一个 WAR 文件，如下图所示。

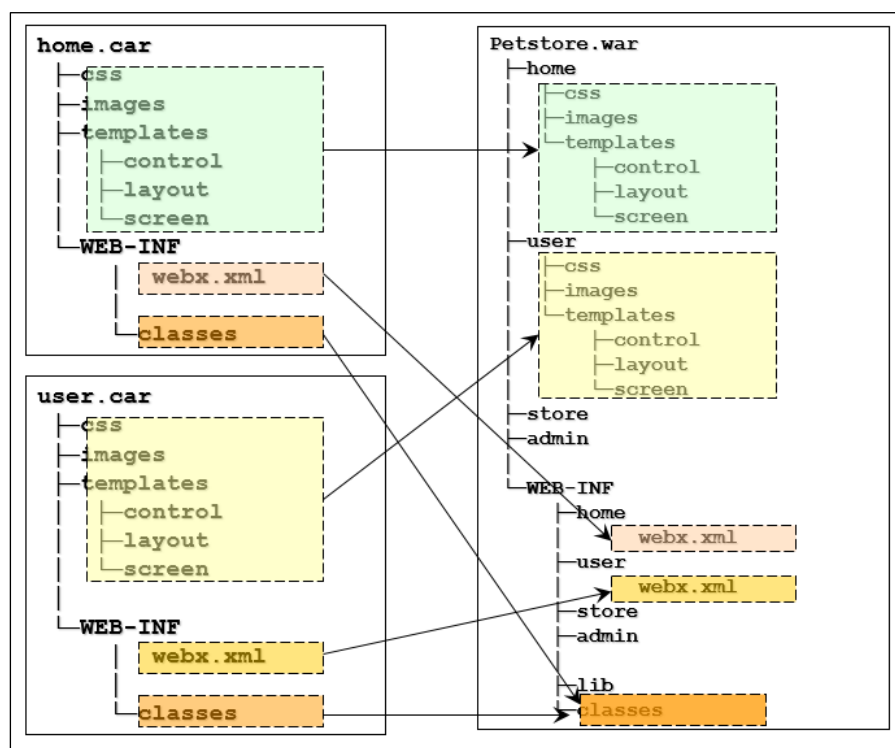


图 2.5-1：多个CAR被合并成一个WAR文件

第3章 应用部署

TongWeb 管理控制台支持文件部署和目录部署两种部署源类型，以及控制台部署、自动部署、热部署和命令行部署等 4 种部署方式。

3.1 部署源的两种类型

3.1.1 文件部署

文件部署即应用以应用包（如*.war, *.ear 等）的形式进行部署，该形式的部署支持所有类型的应用。部署后的信息默认存放在`${TongWeb_HOME}/deployment`目录中。

3.1.2 目录部署

目录部署即应用以展开的目录形式进行部署，目录部署支持 Web 应用、EJB 应用、Connector 和 EAR 应用等各种类型的应用。目录部署的优点是方便应用的修改，当应用包含了需要频繁修改的文件时，使用目录部署会相对方便。

注意：

EAR 应用在使用目录部署时，如果子模块中包含 Web 应用，需要将其解压为以.war 或_war 结尾的目录，否则无法识别该子模块。子模块中的 EJB 应用可以不解压，如果也要解压，其目录需要以.jar 或_jar 结尾。

3.2 应用部署的四种方式

TongWeb 应用部署的 4 种方式，包括控制台部署、自动部署、热部署和命令行部署。控制台部署是通过管理控制台上的“应用管理”模块对应用进行部署；自动部署和热部署是通过“服务”模块对应用开启自动部署；命令行部署是通过命令行对应用进行部署。

在 TongWeb 上部署应用时，建议根据应用包的大小来选择部署方式。1GB 以下的优先使用管理控制台部署；1GB 以上的推荐使用自动部署（注：建议使用目录部署源）。对于没有管理控制台的轻量级版 TongWeb，则只能使用命令行部署。

3.2.1 控制台部署

TongWeb 管理控制台支持多种类型的文件部署，通过管理控制台页面上的“**部署应用**”，对应用进行部署的操作步骤如下。

1. 在管理控制台的左侧导航栏中单击“应用管理”后，进入应用列表页面，如下图所示。



图 3.2-1: 部署应用

2. 单击“部署应用”按钮，进入“应用管理”界面，文件位置分为本机和服务器，如下图所示。



图 3.2-2: 选择部署文件

3. 如果部署文件在本机，会将本地文件例如下图中的TC_examples.war上传到服务器上，如下所示。



图 3.2-3: 上传部署文件

说明:

TC examples.war是一个用于练习本机操作的范例文件，位于

`\TongWeb HOME\samples\servletjsp-samples\servletjsp-tomcatexamples\TC examples.war`

4. 如果部署文件已经在服务器上，直接选择服务器上对应的应用文件或其解压后的应用目录，如下图所示。

说明：

左侧显示为在服务器上启动TongWeb的用户所在的根目录，例如/home/xxx，以及在管理控制台首页控制台配置中添加的服务器选择文件可选目录，例如下图中的/home。

注意：

服务器部署需要先去首页配置服务器选择文件可选目录，首次登录不提供修改功能，用到需要选择文件的组件，页面会给出提示，需要自己手动去单机控制台首页、设置服务器选择文件可选目录；目录的设置要求输入绝对路径，

windows不允许设置为系统盘根目录、不允许为系统盘的Windows目录；linux不允许设置为"/"、"/root"、"/etc"。



图 3.2-4：在服务器上选择部署文件

5. 单击“开始部署”按钮，进入“基本属性”界面，如下图所示。

The screenshot shows the '应用管理' (Application Management) interface. At the top, there's a header '应用管理 管理应用' and a sub-header '部署应用--TC_examples'. Below this is a progress bar with three steps: '1 基本属性' (Basic Properties), '2 虚拟主机设置' (Virtual Host Settings), and '3 完成部署!' (Complete Deployment!). The 'Basic Properties' tab is active, showing various configuration fields for the application 'TC_examples'. Fields include '应用名称' (Application Name), '应用前缀' (Application Prefix), 'session管理器' (Session Manager), '部署顺序' (Deployment Order), 'JSP预编译' (JSP Pre-compilation), '请求参数丢失校验' (Request Parameter Loss Check), '共享库' (Shared Library), '类加载顺序' (Class Loading Order), '描述' (Description), '应用密级' (Application Security Level), '防篡改' (Anti-tampering), '部署超时' (Deployment Timeout), and '类热加载' (Class Hot-loading). Each field has a corresponding description or default value. For example, '应用名称' is 'TC_examples', '应用前缀' is '/TC_examples', 'session管理器' is '无', '部署顺序' is '100', 'JSP预编译' is '支持', '请求参数丢失校验' is '否', '共享库' is '请选择', '类加载顺序' is '父优先', '描述' is empty, '应用密级' is '普通', '防篡改' is '否', '部署超时' is '600', and '类热加载' is '否'. At the bottom, there are buttons for '重新上传' (Re-upload), '下一步' (Next Step), and '取消' (Cancel).

图 3.2-5：设置应用基本属性

所有基本属性的作用如下，本案例中所有基本属性保持默认即可。

- **应用名称：**默认情况下，应用名称就是部署时选择的应用文件或者目录名，这个名称在部署阶段可以修改。
- **应用前缀：**应用上下文根路径，对应的是Web应用。每个 Web应用对应一个访问前缀，当访问具体Web应用时作为应用名出现在请求URI中。例如，部署Web应用时指定访问前缀为"test"，则所有对该应用下JSP页面和Servlet的请求形式必须为“http(或https)://服务器IP:监听端口/test/.....”。

如果访问前缀为"/"则表示访问前缀为空。每个EAR应用中Web模块的访问前缀，由Web应用的自定义部署描述文件tongweb-web.xml中的属性“context-root”决定，或由META-INF/application.xml中“web-uri”决定（注：应用前缀在application.xml中优先级高于tongweb-web.xml）。

如果没有上述相应的配置，则使用Web应用包名（文件部署，不含.war扩展名）或目录名（目录部署）作为访问前缀。在EAR应用中，每一个 Web应用都要有一个唯一的访问前缀，任何两个Web应用不能有相同的访问前缀。

- **session 管理器：**在下拉列表中选择已创建的 session 管理器，默认为“无”，表示使用内存类型的 session 管理器。Session 管理器支持内存、TDG、redisSession 以及自定义类型的 session

管理器。

- **部署顺序**：默认值 100，根据值的高低决定 TongWeb 重新启动后应用部署的先后顺序。
- **JSP预编译**：在管理控制台部署应用时，可以设置当前应用的JSP预编译。设置完成后，对于 Web应用和EAR应用中的Web模块，都会首先调用应用服务器内置的JSPC工具。随着应用的部署，JSP会进行预编译，避免在第一次访问时再进行编译。这个属性的作用是提高请求JSP的访问速度。
- **请求参数丢失校验**：默认为 false。当设置为 true 是，访问应用时请求参数，超过最大参数个数，则返回 400 异常；post 请求时的参数，超过请求最大字节数，则放回 413 异常。
- **共享库**：应用类型为War、Ear时，可以给当前应用配置共享库，支持多选。选择共享库之后，可以手动检测出所选共享库中的多版本以及冗余类，同时可以排除一些加载包，这样就能有效的避免应用运行时发生某些错误。如果共享库引用的是文件夹，则会加载文件夹下面所有的JAR包进行检测。如果未排除任何冗余或者多版本，应用将加载所选共享库下的全部加载包（按路径去除重复）。应用类型为Ear时，配置的共享库会作用于该应用的所有子模块。

注：共享库需要预先配置，请参见 “[共享库](#)” 章节。

- 1) 当应用部署类型为War、Ear时，在部署时可以选择共享库。共享库中分为私有库和公有库，公有库为所有应用共享，不能对其进行排除。如果只有这个应用自己加载，则选择私有库。如下图：

图 3.2-6：应用共享库

- 2) 选择完共享库后，点击后面的“多版本或冗余类检测”，进入类检测界面。可对不需要的类点击后面的“排除”。如下图所示。

多版本或冗余类检测					
多版本(0) 冗余类(3) 已排除(0) 运行时(3)					
来源	类型	名称	版本	路径	操作
com.mengal.test.SimpleBean					
private1	私有	--	--	/home/zhuohq/tw7/common/dir1/HelloWorldJar.jar	排除
private2	私有	--	--	/home/zhuohq/tw7/common/dir2/HelloWorldJar2.jar	依赖
com.mengal.test.HelloWorldBeanLocal					
private1	私有	--	--	/home/zhuohq/tw7/common/dir1/HelloWorldJar.jar	排除
private2	私有	--	--	/home/zhuohq/tw7/common/dir2/HelloWorldJar2.jar	依赖
com.mengal.test.HelloWorldBean					
private1	私有	--	--	/home/zhuohq/tw7/common/dir1/HelloWorldJar.jar	排除
private2	私有	--	--	/home/zhuohq/tw7/common/dir2/HelloWorldJar2.jar	依赖

图 3.2-7：多版本或冗余类检测

多版本：名称一样，版本不同，名称和版本取值于加载包中的META-INF/MANIFEST.MF内容。

冗余类：同一个类(包名+类名)在多个加载包中存在。冗余类能够被真正加载到内存里的只有一份，具体要加载哪个由java虚拟机决定，冗余类可能会导致加载到错误的类（如版本错误）。

已排除：表示排除的多版本或者冗余类，新的排除项需要点击重新检测才能显示，已排除的包支持重新依赖。不能排除应用依赖以及公有库依赖包，只能排除私有库里面的包。

运行时：显示当前应用运行时加载的依赖包，不能执行操作，新的排除项需要点击重新检测才能显示。

重新检测：排除某些多版本或者冗余类加载包之后，点击重新检测，系统根据当前排除项重新计算多版本或者冗余，每个类别上都有数字提示当前类别的数量，可根据数量直观的判断出是否存在多版本或者冗余类。

- **类加载顺序：**类加载顺序是指定Web应用、EAR应用中Web模块和EJB模块的类加载顺序，分为父优先和子优先两种。父优先即在加载类的时候，在从类加载器自身的类路径上查找加载类之前，首先尝试在父类加载器的类路径上查找和加载类；子优先即在加载类的时候，首先尝试从自己的类路径上查找加载类，在找不到的情况下，再尝试父类加载器的类路径。

注意：

通过管理控制台部署EAR应用时，可以为EAR应用中所有的Web模块指定同样的类加载顺序，也可以在编辑时单独为每个Web模块设置不同的类加载顺序。

管理控制台指定的类加载顺序优先于应用中tongweb-web.xml部署描述文件指定的类加载顺

序，tongweb-web.xml中的类加载顺序优先于其他配置文件的类加载顺序。

- **描述：**应用描述用来注明该Web应用项目的描述信息，每一个应用都默认在\${TongWeb_HOME}/conf的tongweb.xml中存储着关于这个应用的描述。
- **应用密级：**TongWeb对应用提供“普通、机密、绝密”三种等保密级，可在应用部署时对应用的密级标识进行选择。
- **防篡改：**开启防篡改功能，可对核心文件进行防篡改设置，及时发现和恢复被篡改文件，防止应用核心文件被意外、恶意篡改，为应用提供安全防护。
- **部署超时：**部署超时时间，部署超时后，中断部署，打出堆栈。默认值为“600”，取值范围为“1-3600”，单位：秒。
- **类热加载：**开启类热加载后，在TongWeb服务器启动且应用在运行过程中，自动检测应用的class文件更新。若class文件存在更新，则更新的class文件立即生效，无需重启TongWeb服务器。
若您需要TongWeb本身的类支持热加载，可在启动参数中添加“-DenableHotSwap=true”参数，开启TongWeb自身类的热加载，默认为关闭，并将需要热加载的class文件放置在“\${TongWeb.home}/lib/hotswap-classes”目录下。为了避免类冲突，需要class文件的完整包路径，例如：com/example/test/Test.class，没有类的包目录或包目录错误会导致此class的热加载不生效。

注意：

- 新增或者删除的class文件不会触发热加载。
- 应用停止或解部署后，TongWeb服务器停止应用的类热加载的监听。
- 生产环境请谨慎开启。
- 类热加载不支持变更了结构类的加载，只支持热加载修改类方法内部的代码。

6. 应用的基本属性设置完成后，单击“下一步”进入“**虚拟主机设置**”界面，单击下拉列表选择“server”虚拟主机，如下图所示。虚拟主机只能用于Web应用与含有Web模块的EAR应用。同一个应用可以部署到多个虚拟主机上，通过在DNS配置的主机名称，在浏览器中进行指定的访问。



图 3.2-8：虚拟主机设置

7. 单击“下一步”进入“完成部署”界面，汇总确认前面填写的信息，如下图所示。如果想对基本属性进行更改，单击“上一步”返回即可。

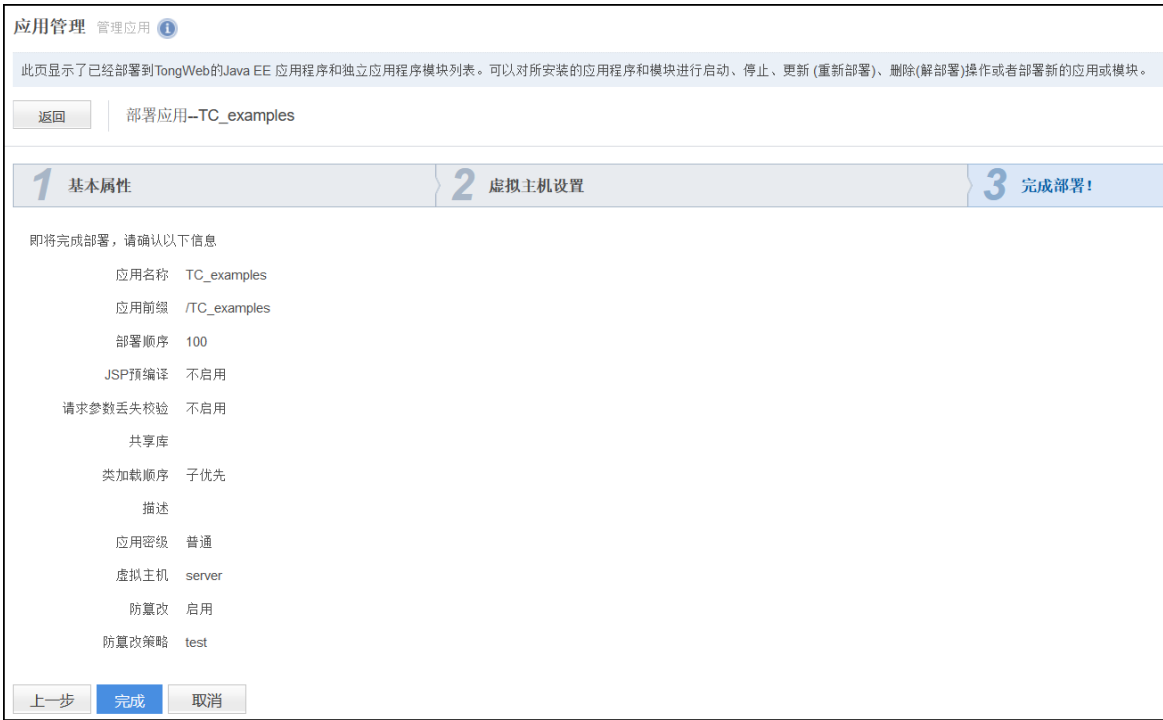


图 3.2-9：确认部署信息

8. 确定基本属性设置的信息后，单击“完成”按钮。如果没有出现任何异常时，说明应用部署成功。在已部署的应用页面可查看该应用已经部署成功，如下图所示。



图 3.2-10：应用部署成功

注意：

如果部署过程中出现异常的信息，即应用部署出现错误。页面会跳转到查看已部署的应用页面，在页面最上端出现部署失败的提示框，如下图所示。单击提示框中的“了解详情”链接，弹出下载文件提示，文件里面包含显示异常的详细信息，可用来分析本次部署失败的原因。



图 3.2-11：应用部署失败

3.2.2 自动部署

TongWeb支持文件和目录两种方式的自动部署，当启动自动部署功能后，TongWeb会对自动部署目录进行监控，自动执行应用的部署和解部署操作。

3.2.2.1 自动部署设置

自动部署的启动和停止可通过TongWeb管理控制台进行配置。进入管理控制台，选中“服务”模块，如下图所示。

服务配置

配置服务

此页显示了服务配置信息，您可以更新这些配置，并重新启动服务器使其生效。

部署服务设置

自动部署

☒ 开启

开启之后监听的是目录中应用的增加和删除

自动部署目录

{serverDir}

Agent

applications

autodeploy

bin

conf

deployment

domain_template

lib

license_update

logs

patch

patchManage

persistence

已选择文件\${tongweb.root}\autodeploy\

时间间隔

3000

毫秒

检查自动部署目录的时间间隔，以毫秒为单位。默认为3000毫秒

部署超时时间

600

秒

部署耗时超过此时间，将中断部署流程。默认600秒

热部署

☐ 开启

是否开启部署目录资源修改后实时生效的开关

自动部署虚拟主机

server

从列表中选择自动部署的虚拟主机

服务模块配置

图 3.2-12：开启自动部署

在服务配置页面中，可以设置自动部署的以下4个选项。

- **自动部署：**选中该属性，则启动自动部署功能；取消选中，则停止自动部署功能。
注意：
在修改了该属性后，需要重启服务器才可以生效。
- **自动部署目录：**可以选择在服务器的任何一个目录进行自动部署，默认自动部署目录为 **`${TongWeb_HOME}/autodeploy`**。
- **时间间隔：**自动部署目录的监控线程扫描时间，以毫秒为单位。
- **部署超时时间：**自动部署耗时超过此时间，系统将中断部署流程，默认 600 秒。
- **自动部署虚拟主机：**从列表中选择自动部署的虚拟主机。默认 **server**。

上述选项设置完成后，单击服务配置页面下方的“保存”按钮，即可启动自动部署的功能。

3.2.2.2 自动部署应用的方法

TongWeb支持文件和目录两种方式的自动部署。

自动部署应用需要将部署的应用文件或目录复制到自动部署目录后即可，支持的文件形式包括EAR应用（.ear）、Web 应用（.war）和EJB 应用（.jar）。例如，某个EAR应用的目录名称为enterprise，那么直接将enterprise目录放入自动部署目录进行自动部署。

部署操作结束后，在自动部署目录中的\${TongWeb_HOME}/autodeploy/.autodeploystatus文件夹下会出现一个对应部署应用的部署状态文件夹。这是个空的文件夹，只是用来标志部署操作是否成功。

当出现上述文件夹，并且文件夹名称与应用名称相同时，说明自动部署应用成功，当.autodeploystatus 文件夹下出现“应用名.failed”的时候，说明该应用自动部署失败。

注意：

如果通过自动部署的方式部署带版本的应用，会将其作为普通应用来部署。自动部署应用的类加载顺序使用应用的自定义部署描述文件中配置的顺序。如果应用中无自定义部署描述文件或该文件未指定类加载顺序，默认使用子优先。

3.2.3 热部署

热部署同样是应用部署的一种方式，其主要的功能是在应用部署之后，在线实时对应用进行修改，结果会立刻展现。

3.2.3.1 热部署设置

开启热部署需要通过TongWeb管理控制台进行设置。

1. 进入管理控制台。
2. 在左侧导航栏中，选择“服务”，进入服务配置页面。
3. 开启“热部署”选项后，即可开启热部署。

服务配置

配置服务

此页显示了服务配置信息，您可以更新这些配置，并重新启动服务器使其生效。

部署服务设置

自动部署

☐ 开启

开启之后监听的是目录中应用的增加和删除

自动部署目录

{serverDir}

📁

lib_202503-10-10-00

📁

lib_202503-10-10-01

📁

lib_202503-10-10-02

📁

lib_202503-10-10-03

📁

lib_202503-10-10-04

📁

lib_202503-10-10-05

已选择文件: \${tongweb.root}\autodeploy\

时间间隔

3000

毫秒

检查自动部署目录的时间间隔，以毫秒为单位。默认为3000毫秒

热部署

☒ 开启

是否开启部署目录资源修改后实时生效的开关

自动部署虚拟主机

server

从列表中选择自动部署的虚拟主机

图 3.2-13: 开启热部署

3.2.3.2 热部署应用的方法

当热部署的开关开启后，修改`/${TongWeb_HOME}/deployment`中的应用的类文件或应用的配置文件`web.xml`后，会触发整个应用进行重新部署。应用重新部署后，将最新的内容提供给用户。

如果热部署的是EAR应用，则只需要替换deployment下该EAR应用目录下未解压的子模块（Web 模块或EJB模块），该EAR应用就会发生热部署。

注意：

替换EAR目录下已经解压的子模块中的内容，不会触发热部署。

3.2.4 命令行部署

TongWeb支持命令行部署，命令行的启动脚本位于\${TongWeb_HOME}/bin目录下，**commandstool**命名的脚本，有bat（Windows）和sh（Linux/Unix）两个版本。**commandstool**的执行方式及基本参数请参见《006 TongWebCommandstool使用指南》，此处提供部署应用命令的使用说明。

使用commandstool定义的deploy命令，可以部署Web 应用、EJB应用及企业应用等。

```
commandstool>deploy --contextroot=ejb3_1_war --precompilejsp=true
--applocation=C:\\TongWeb\\samples\\ejb\\ejb3_1_war.war testapp1
```

相关参数说明:

- **--applocation:** 客户端应用文件的路径，必填（注：windows下路径使用“\\”或者“/”）

- **--defaultvs:** 虚拟服务器
- **--contextroot:** 应用前缀。只有在web应用部署时可用。
- **--precompilejsp:** JSP是否预编译
- **--deployorder:** 设置部署顺序
- **--appdescription:** 应用描述
- **--delegate:** 类加载策略，默认是子优先false，如果想配置父优先则设置为true。
- **目标参数:** 应用名称（例如：testapp1/testapp2），必填。

第4章 应用管理

TongWeb的应用管理包括使用管理控制台、自动扫描、命令行管理和接口管理等4种方式。

4.1 使用管理控制台

应用管理最方便的途径是通过管理控制台的“应用管理”模块进行统一的管理，包括：查看已经部署的应用；部署应用；查看EAR应用的子模块；应用的解部署、重部署、编辑、访问；应用的启动和停止。

本节进行的操作都在应用管理页面进行。在管理控制台左侧的导航栏中单击“应用管理”即可进入应用管理页面。

4.1.1 查看应用

应用部署完成后，即可在应用管理界面查看所有已经部署的应用，并以列表的形式展现，如下图所示。



名称	前缀	应用类型	部署源类型	部署方式	虚拟主机	状态	操作
testapp2	/ejb3_1_war	war	文件部署	控制台部署	server	已启动	重部署 http访问 https访问 备份
ejb2		ear	文件部署	控制台部署	server	已启动	重部署 子模块 备份
TC_examples	/TC_examples	war	文件部署	控制台部署	server	已启动	重部署 http访问 https访问 备份
simple-cmp-war	/simple-cmp-war	war	文件部署	控制台部署	server	出错	重部署 http访问 https访问 备份
genericra		rar	目录部署	控制台部署	server	已启动	重部署

图 4.1-1：应用列表

列表显示了“部署应用”时设置的名称、前缀、虚拟主机等基本属性，应用类型、部署源类型及部署方式。（注：基本属性说明可参见“[控制台部署](#)”章节）

列表也显示了应用的部署状态，包括正在部署、已启动、已停止、未退休、已退休、退休中、出错。

列表还提供了对各个应用的操作：包括http访问、重部署、子版本、子模块、备份。

- **http/https访问**：单击可以访问应用，若应用部署的虚拟主机所关联的通道均为空，则该超链接显示为灰色不可用；
- **重部署**：当应用部署完成后，如果发现应用中的某些类或配置不正确，可以重新部署应用；
- **子版本**：可以查看应用所有的版本，仅适用于带版本的应用；
- **子模块**：可以查看应用所有的子模块，仅适用于带子模块的应用；
- **备份**：可以备份当前应用。

4.1.2 编辑应用

在查看完所有已部署应用后，如果想对应用的基本属性进行修改，可以对其进行编辑。

1. 在应用管理界面，单击“名称”列下面的应用名，例如Web应用“testweb”，进入应用查看和编辑页面。如下图所示，对基本属性进行编辑后，单击“保存”按钮更新应用的信息。

返回

编辑应用-- TC_examples

基本属性

应用名称	TC_examples	应用名称
应用位置	\${tongweb.home}/deployment/TC_examples	应用位置
原始应用包路径	\${tongweb.home}/temp/upload /TC_examples.war	原始应用包路径
应用前缀	/TC_examples	应用前缀
部署顺序	100	默认的部署顺序是100，如果需要调整部署顺序的话，可以指定
JSP预编译	☐ 支持	JSP预编译
请求参数丢失校验	☐	请求参数丢失校验
共享库	请选择	应用运行时使用的共享类库 多版本或冗余类检测
类加载顺序	☐ 父优先 ☒ 子优先	类加载顺序，默认为子优先，可调整
描述		该应用的描述信息
应用密级	☒ 普通 ☐ 机密 ☐ 绝密	应用的密级标识信息
防篡改	☒ 开启	是否开启应用防篡改
防篡改策略	test	防篡改策略必选一项

图 4.1-2：编辑Web应用

2. 对于EAR应用来讲，此编辑页面的类加载顺序以树状形式展示、可编辑web模块的类加载顺序，保存后更新应用信息，如下图所示。

返回

编辑应用-- ejb2

基本属性

应用名称	ejb2	应用名称
应用位置	\${tongweb.home}/deployment/ejb2	应用位置
原始应用包路径	\${tongweb.home}/temp/upload/ejb2.ear	原始应用包路径
部署顺序	100	默认的部署顺序是100，如果需要调整部署顺序的话，可以指定
JSP预编译	☐ 支持	JSP预编译
请求参数丢失校验	☐	请求参数丢失校验
共享库	请选择	应用运行时使用的共享类库 多版本或冗余类检测
类加载父优先	☒ ejb2	企业应用ejb模块类加载顺序的指定，选中复选框即指定为父优先
描述		该应用的描述信息
应用密级	☒ 普通 ☐ 机密 ☐ 绝密	应用的密级标识信息
防篡改	☒ 开启	是否开启应用防篡改

图 4.1-3：编辑EAR应用

4.1.3 访问应用

对应用的基本属性编辑确认完成后，就能对应用进行访问。可访问的应用类型包括WAR和EAR两种，WAR应用通过应用列表中的访问链接即可进行访问；EAR应用的访问链接在子模块页面的操作栏中。

访问WAR应用

WAR应用的访问链接在已部署应用列表的操作栏下。

单击WAR应用“操作”列下面的“**http访问**”或“**https访问**”，即可在弹出的新窗口中对该应用进行访问。

注意：

如果部署应用的虚拟主机上关联了HTTPS通道，https访问的链接才可用。

访问EAR应用

EAR应用存在子模块，其访问的链接在子模块页面的操作栏中。

1. 在EAR应用ejb2的“操作”列中，存在“**子模块**”的链接，如下图所示。

应用管理 管理应用 1								
此页显示了已经部署到TongWeb的Java EE 应用程序和独立应用程序模块列表。可以对所安装的应用程序和模块进行启动、停止、更新(重新部署)、删除(解部署)操作或者部署新的应用或模块。								
部署应用 启动 停止 解部署 设置应用备份路径 搜索								
☐	名称	前缀	应用类型	部署源类型	部署方式	虚拟主机	状态	操作
☐	testapp2	/ejb3_1_war	war	文件部署	控制台部署	server	已启动	重新部署 http访问 https访问 备份
☐	ejb2		ear	文件部署	控制台部署	server	已启动	重新部署 子模块 备份
☐	TC_examples	/TC_examples	war	文件部署	控制台部署	server	已启动	重新部署 http访问 https访问 备份
☐	simple-cmp-war	/simple-cmp-war	war	文件部署	控制台部署	server	出错	重新部署 http访问 https访问 备份
☐	genericra		rar	目录部署	控制台部署	server	已启动	重新部署

图 4.1-4：应用列表

2. 单击“**子模块**”的链接，弹出子模块的列表页面，单击操作列下面的“**http访问**”或“**https访问**”，即可弹出新窗口对该应用进行访问，如下图所示。

子模块		
名称	类型	操作
HelloBean	ejb	
JobProcessorBean	ejb	
TimerBean	ejb	
PropertyRegistryBean	ejb	
web.war	war	http访问 https访问
		取消

图 4.1-5：EAR应用的子模块信息

4.1.4 停止应用

应用部署完成后，默认为“已启动”的状态，这个时候可以对应用进行停止。如下图所示，在列表选中对应的应用，其应用状态为“已启动”状态。单击列表上方的“**停止**”按钮，即可将其停止。

注意：

当停止应用时，容器调用servlet、filter等的destory方法，以便让Servlet相关对象可以释放它所使用的资源。如应用新建的端口，线程需要在destory方法中声明销毁。

当启动应用时，相当于进行了部署流程。

应用管理 管理应用 1

部署应用

启动

停止

解部署

设置应用备份路径

搜索

☐	名称	前缀	应用类型	部署源类型	部署方式	虚拟主机	状态	操作
☐	testapp2	/ejb3_1_war	war	文件部署	控制台部署	server	已启动	重部署 http访问 https访问 备份
☐	ejb2		ear	文件部署	控制台部署	server	已启动	重部署 子模块 备份
☒	TC_examples	/TC_examples	war	文件部署	控制台部署	server	已启动	重部署 http访问 https访问 备份
☐	simple-cmp-war	/simple-cmp-war	war	文件部署	控制台部署	server	出错	重部署 http访问 https访问 备份
	genericra		rar	目录部署	控制台部署	server	已启动	重部署

控制台

应用 TC_examples 停止成功

应用管理 管理应用 1

部署应用

启动

停止

解部署

设置应用备份路径

搜索

☐	名称	前缀	应用类型	部署源类型	部署方式	虚拟主机	状态	操作
☐	testapp2	/ejb3_1_war	war	文件部署	控制台部署	server	已启动	重部署 http访问 https访问 备份
☐	ejb2		ear	文件部署	控制台部署	server	已启动	重部署 子模块 备份
☒	TC_examples	/TC_examples	war	文件部署	控制台部署	server	已停止	重部署 http访问 https访问 备份
☐	simple-cmp-war	/simple-cmp-war	war	文件部署	控制台部署	server	出错	重部署 http访问 https访问 备份
	genericra		rar	目录部署	控制台部署	server	已启动	重部署

图 4.1-6：停止应用

4.1.5 启动应用

当应用处于停止状态时，可再次启动应用。在列表选中对应的应用，其应用状态为“已停止”状态。单击列表上方的“**启动**”按钮，即可将其启动，重新允许访问应用。如下图所示。



图 4.1-7：启动应用

4.1.6 解部署应用

在应用列表中显示的是所有已经部署完成的应用，如果想要删除应用列表中的应用，可以对应用进行解部署。

如下图所示，勾选需要解部署应用左侧的复选框，选中对应的应用，然后再单击列表上方的“解部署”按钮，弹出确认删除提示对话框，单击“删除”按钮，应用被解部署。



图 4.1-8：解部署应用

4.1.7 重部署应用

当应用部署完成后，如果发现应用中的某些类或配置不正确，修改后则需要重新部署应用。

- 如下图所示，修改已部署应用目录下的应用后，单击“操作”列下面的“重部署”按钮，弹出重部署应用的页面，单击“确定”按钮后，对应的应用即进行重部署。



图 4.1-9：重部署应用

2. 勾选“使用新的部署文件”还可以使用新的部署文件进行重部署，如下图所示。



图 4.1-10：选择新的重部署文件

注意：

“使用新的部署文件”相当于销毁旧应用、重新部署一个新应用。这一步的操作和部署的页面类似，但是原应用的“部署基本属性”在新应用中被继承下来，而不能进行选择。从这一点可以看到，如果当前重部署的新应用与原应用从应用的形态上相差过大，可能会导致应用重部署失败。另外，新旧应用的应用类型必须保持一致。

使用原来的部署文件部署，类加载顺序与原来的类加载顺序保持一致；使用新的部署文件部署，类加载顺序与新的部署文件的自定义部署描述文件中配置的一致。如果部署文件中无自定义部署描述文件或该文件未指定类加载顺序，默认使用子优先。

4.1.8 备份应用

在应用列表中显示的是所有已经部署完成的应用，如果想要备份应用列表中的应用，可以对应用进行备份。

如下图所示，单击“操作”列下面的“备份”按钮，在弹出的对话框中单击“确定”按钮后，对应的应用即进行备份。



图 4.1-11：备份应用

应用备份成功后，可点击“了解详情”查看备份文件所在服务器位置，如下图所示。



图 4.1-12：备份应用成功

如想修改应用备份路径，可点击“设置应用备份路径”对应用备份目录进行修改。应用备份路径可以设置绝对路径或\${TongWeb_HOME}下的相对路径，修改成功后再选择备份应用会拷贝到新目录下。

4.1.9 更新应用

更新应用是带有版本的应用对自身版本升级的一种手段，当前仅支持Web应用的更新。更新的内容包括跟随着应用发布的JNDI对应资源节点以及Web对应的资源。

首先需要准备好新版本应用，在应用目录包下的META-INF/MANIFEST.MF文件中定义属性名为**Tongweb-App-Version**；属性值为版本号，采用X.Y的形式，其中X和Y都是数字（比如1.0、1.1、1.2等等诸如此类）。例如：Tongweb-App-Version: 1.3。

当带版本信息的应用准备完成后，按照以下的几个步骤在管理控制台中更新应用。

- 如下图所示，单击左上角的“部署应用”按钮，将带版本的应用文件JasperJsp(versioned)部署到TongWeb上。（注：该应用文件为用户自行开发的应用示例，TongWeb安装包的samples不提供带版本的应用。）



图 4.1-13：部署带版本的应用文件

2. 当JasperJsp(versioned)需要更新时，单击“操作”列的“子版本”，进入应用版本管理页面，如下图所示，该页面显示带版本应用的名称、前缀、应用类型、虚拟主机、退休状态和版本号等信息。



图 4.1-14：子版本列表

3. 选择要更新的应用，单击列表上方的“更新”按钮，进入更新应用界面，如下图所示。



图 4.1-15：选择更新应用文件

4. 将新版本文件上传到服务器端，单击“下一步”按钮，填写应用附加属性页面。在附加属性页面中，增加了更新策略：**自然退休**和**强制退休**。自然退休即当前应用不再接受新的请求，等待老的请求（已经建立了会话的请求）的会话超时之后再进行退休；强制退休是立即退休，选中“强制退休”选项之后，可以加上一个强制退休时间作为备选选项，如下图所示。



图 4.1-16：更新应用附加属性

5. 应用更新结束后，页面跳转到应用版本管理页面。应用已经更新完成，列表中存在两个应用，名称和前缀相同，但是版本号不同。被更新的应用的状态为已退休或者正在退休，而更新的应用的状态为未退休，如下图所示。

应用JasperJsp的所有版本 应用版本管理 ⓘ						
此页显示了该应用的所有版本						
返回 更新						
名称	前缀	应用类型	虚拟主机	退休状态	版本号	操作
☐ JasperJsp	/JasperJsp	war	server	已退休	1.3	切换到该版本 删除该版本
☐ JasperJsp	/JasperJsp	war	server	未退休	1.4	

图 4.1-17：子版本列表

6. TongWeb对“已退休”的应用版本，还提供了切换和删除的功能。单击“切换到该版本”，可以将当前的应用切换到不同的版本，单击“删除该版本”可以将对应版本的应用删除。

4.2 自动扫描

自动扫描为应用管理的一种形式，分为自动解部署和自动重部署。

4.2.1 自动解部署

要将已经自动部署的应用进行解部署，可直接删除部署时复制到自动部署目录 `${TongWeb_HOME}/autodeploy` 下的应用文件。自动解部署结束后，`${TongWeb_HOME}/autodeploy/.autodeploystatus` 文件夹下面的应用同名文件夹也会被自动删除。如果上述文件夹被删除，说明应用已经成功解部署。

如果自动部署应用失败，进行解部署后，过一段时间查看 `.autodeploystatus` 文件夹，`.failed` 文件也会被自动清除。

4.2.2 自动重部署

当自动部署应用出现错误，即\${TongWeb_HOME}/autodeploy/.autodeploystatus文件夹出现“应用名.failed”文件时，对此应用可以进行重部署。

将修改过的相同应用文件或目录重新复制到自动部署目录\${TongWeb_HOME}/autodeploy下，自动重部署会直接替换部署目录\${TongWeb_HOME}/deployment下对应的原应用文件和目录。

如果应用是通过文件部署的，那么是否进行重部署是根据应用文件的大小和修改时间进行判定的，如果新的应用文件和之前部署的应用文件大小和修改时间都相同，则不会进行重部署。

注意：

针对目录部署的情况，是根据同级文件夹的大小和修改时间判定的。

4.3 命令行管理

通过命令行工具也可以对应用进行本地或远程的部署，解部署，重部署及查看已部署应用等。

4.3.1 deploy

表 4.3-1: deploy命令说明

功能说明	部署企业应用、Web 应用、EJB 模块。
相关参数	● --aplocation: 必选，客户端应用文件的路径。 ● --contextroot: 应用前缀。只有在web应用程序部署时可用。 应用前缀的优先级由高到低依次是：“-contextroot参数” > “tongweb-web.xml” > “默认值（包名）”。 ● --defaultvs: 虚拟服务器。 ● --precompilejsp: jsp是否预编译。默认为“false”，可设置为“true”或者“false”。 ● --requestparameterslostvalidation:默认为“false”，请求参数丢失校验参数。可设置为“true”或者“false”，true表示校验参数，false表示不校验。 ● --deployorder: 设置部署顺序。默认的部署顺序是100，如果需要调整部署顺序的话，可以指定。取值范围为“1-2147483647”之间的整数。 ● --delegate: 类加载策略，默认是子优先“false”，如果想配置父优先则设置为“true”。 ● --appdescription: 应用描述。

	● <code>--sharedLibNames</code>: 指定共享库名称, 可以指定多个, 以 “,” 分隔。 ● <code>--enableHotSwap</code>: 开启类热加载, 默认为 “false”。当设置为 “true” 时, 在TongWeb服务器启动且应用在运行过程中, 自动检测应用的class文件更新。若class文件存在更新, 则更新的class文件立即生效, 无需重启TongWeb服务器。 注意: ■ 新增或者删除的class文件不会触发热加载。 ■ 应用停止或解部署后, TongWeb服务器停止应用的类热加载的监听。 ■ 生产环境请谨慎开启。 ■ 类热加载不支持变更了结构类的加载, 只支持热加载修改类方法内部的代码。 ● <code>--deployTimeout</code>: 部署超时时间, 若部署超过此时间, 系统将中断部署流程, 默认600秒。 ● <code>--sessionManager</code>: 指定session管理器的名称。 ● <code>app_name</code>: 必选, 目标参数, 应用名称。
命令行及实例	本地部署: <pre>deploy --contextroot=/dbpooltest --precompilejsp=true --applocation=C:\\test\\dbpooltest.war testapp</pre> 注: windows 的路径使用“\\” 或者“/”
注意事项	deploy 命令只支持本地主机进行文件或目录部署。
验证方式	在管理控制台的应用管理下的应用列表中查看, 或用 <code>list-apps</code> 查看, <code>testapp</code> 是否部署成功。

4.3.2 undeploy

表 4.3-2: undeploy命令说明

功能说明	解除部署在指定目标上的组件
相关参数	目标参数: 应用名称
命令行及实例	本地解部署: <pre>undeploy testapp</pre>
注意事项	应用名称的默认值是去掉文件后缀的名称, 例如: 应用为 <code>dbpooltest.war</code> , 默认的应用名称为 <code>dbpooltest</code> 。如过部署时更改其应用名称为 <code>testapp</code> , 解部署应用时也需要指定应用名称为 <code>testapp</code> 。如果应用名称不正确, 将导致解部署失败。为了确认应用名称, 可以通过 <code>list-apps</code> 查看是否有当前应用名称存在。

验证方式	在管理控制台的应用管理下的应用列表中查看，或用 list-apps 查看，testapp 是否解部署成功。
------	---

4.3.3 redeploy

表 4.3-3: redeploy命令说明

功能说明	重新部署在指定目标上的组件
相关参数	--applocation: 新的应用文件的路径 目标参数: 应用名称
命令行及实例	本地重部署（不替换应用文件）： redeploy testapp 本地重部署（替换新的应用文件）： redeploy --applocation=C:/test/dbpooltest_new.war testapp
注意事项	1. redeploy 命令只支持本地主机进行文件或目录部署。2. 应用名称的默认值是去掉文件后缀的名称，例如：应用为 dbpooltest.war，默认的应用名称为 dbpooltest。如过部署时更改其应用名称为 testapp，重部署应用时也需要指定应用名称为 testapp。如果应用名称不正确，将导致重部署失败。为了确认应用名称，可以通过 list-apps 查看是否有当前应用的名称存在。
验证方式	在管理控制台的应用管理的应用列表中单击 testapp 后的 Http 访问，查看修改是否生效。

4.3.4 show-backup-appdir

表 4.3-4: show-backup-appdir命令说明

功能说明	查看应用备份目录。
相关参数	-
命令行及实例	commandstool> show-backup-appdir Please enter the admin user name>cli Please enter the admin password>Cli123.com ***** BakUp App dir is : D:\PMM\TW2022\TW_2022-01-13\TW_2022-01-13\app-bak ***** Command show-backup-appdir executed successfully.

4.3.5 update-backup-appdir

表 4.3-5: update-backup-appdir命令说明

功能说明	修改应用备份目录。
相关参数	目标参数: 应用备份目录名称。

命令行及实例	update-backup-appdir app-bak-test
验证方式	点击管理控制台左侧导航树中的“应用管理”->“设置应用备份路径”，应用备份目录已经修改成功。

4.3.6 backup-app

表 4.3-6: backup-app命令说明

功能说明	备份应用。
相关参数	目标参数：应用名称。
命令行及实例	backup-app testapp
验证方式	在\${TongWeb_HOME}/app-bak-test 目录下，testapp 备份成功。

4.3.7 update-auto-deploy-config

功能说明	修改自动部署开关。支持离线模式。
相关参数	● --autoDeployEnabled: 自动部署开关。Boolean 型，默认 true。 ● --autoDeployDir: 自动部署路径。默认空字符串。 ● --autoDeployCheckInterval: 自动部署超时时间，单位：毫秒，默认 3000。 ● --hotDeployEnabled: 热部署开关。Boolean 型，默认“false”。 ● --vsName: 配置自动部署虚拟主机名称。 ● --deployTimeout: 自动部署耗时超过此时间，系统将中断部署流程，默认 600 秒。
实例	● 在线模式： update-auto-deploy-config --autoDeployEnabled=true --autoDeployDir="d:/temp/autodeploy" --autoDeployCheckInterval=4444 --hotDeployEnabled=false --vsName=test-virtual-host ● 离线模式： update-auto-deploy-config --autoDeployEnabled=false --autoDeployDir="d:/temp/autodeploy" --autoDeployCheckInterval=4444 --hotDeployEnabled=false --vsName=xy-virtual-host --offline=true
验证方式	1. 登录 TongWeb 管理控制台。 2. 左侧导航栏中，单击“服务”，进入服务配置页面。 3. 可以查看到对应属性已修改为设置的值。

4.3.8 list-apps

表 4.3-7: list-apps命令说明

功能说明	列出所有部署的应用，支持查看远程部署的应用列表。
相关参数	--host: 用于远程连接，需要连接的远程 IP 地址。默认 localhost --port: 用于远程连接，需要连接的远程端口。默认 9060
命令行	list-apps --host=<ip> --port=<port>
实例	<pre> commandstool> list-apps Please enter the admin user name>cli Please enter the admin password>Cli123.com ***** name=testapp type=war context-root=/dbpooltest vNames=server status=started ***** Command list-apps executed successfully.</pre>

4.4 接口管理

TongWeb提供了JMX和RestFul两种类型的接口，两类接口中都有应用管理的接口，可以通过开发程序调用接口进行应用管理。

第5章 Connector应用

5.1 Connector应用部署

部署应用时，上传rar应用后，进入应用配置页面，可设置应用名称、部署顺序、选择Connector应用关联的线程池（默认为default-thread-pool）及描述和部署目标。如下图所示部署成功后，该应用被部署到所选部署目标上。

图 5.1-1: Connector应用部署

说明：

TongWeb 安装后会自动部署并启动自带的 Connector 应用 **genericra**，并使用默认提供的线程池 **default-thread-pool**。

5.2 Connector应用查看与编辑

在应用列表中显示的是已经部署完成的应用，选择rar应用进行查看，编辑方式同单机一致。

5.3 Connector应用重部署

应用重部署时，如果哪些节点上此应用关联了连接池、托管对象等资源则不允许重部署操作。

5.4 Connector应用解部署

应用解部署时，如果哪些节点上此应用关联了连接池、托管对象等资源则不允许解部署操作。

说明：

TongWeb自带的Connector应用**genericra**不能解部署。

第6章 虚拟目录

应用中的JSP、html和静态资源可以放在虚拟目录中（本地任意目录），其加载优先级如下。

- war中不存在，虚拟目录下存在，加载虚拟目录下的文件。
- war中存在，虚拟目录下不存在，加载war中的文件。
- war和虚拟目录下都存在并且同名的话，加载虚拟目录下的文件。

jsp 输出为null的值时，默认值是true，即输出为null，如果像输出空串需要配置：

```
<property name="jspPrintNull" value="false"/>
```

虚拟目录功能仅限JSP、html和静态资源，JSP引用的class需要在应用的类路径下，使用方式如下。

在tongweb-web.xml文件的根节点下加入如下内容。

```
<propertyname="aliases" value="/aliasPath1=docBase1,/aliasPath2=docBase2"/>
```

如果应用前缀为“/”，则配置如下：

```
<property name="aliases" value="/=D:\virtualdir">;
```

其中，aliasPath1指http请求URL中该资源的访问路径，docBase1是资源所在的绝对目录，如果有多个虚拟目录需要指定，将多个/aliasPathN=docBaseN用逗号隔开即可，具体的运用代码如下。

```
<?xml version="1.0" encoding="UTF-8"?>
<tongweb-web-app>
<property name="aliases" value="/images=D:\Work\mdir\images,/script=D:\Work\mdir\script,
/pages=D:\Work\mdir\pages,/css=D:\Work\mdir\css"/>
</tongweb-web-app>
```

说明：

如果某应用的静态图片的访问url为http://ip:port/appname/images/code.gif，那么其虚拟目录可以配置为/images=D:\Work\mdir\images。

其中/images是请求URL中该资源的访问路径，D:\Work\mdir\images是存放该资源的绝对路径。同理，/script下可以放置js资源，/pages下可以放置jsp资源，/css下可以放置css文件。

第7章 资源

7.1 文件集

文件集功能是维护管理共享库需要的JAR文件，支持客户端上传或服务器选择，服务器选择时可作为JAR文件或文件夹。

展开管理控制台左侧导航树中的“资源”，单击“文件集”，出现如下图所示的“文件集管理”页面。



文件集管理 显示/隐藏详细信息 ⓘ

此页显示了已添加的文件列表(系统内置文件列表consolejars除外)，可以更新、删除所添加文件或者添加新的文件。

☐ 文件集名称	文件来源	文件状态	上传时间
☐ ant-1.9.7.jar	客户端	正常	2020-01-02 10:54:33
☐ ant-1.9.4.jar	客户端	正常	2020-01-02 10:54:27
☐ spring-boot	服务器	正常	2020-01-02 14:24:47
☐ fillet-kafka-2.3.0.jar	服务器	正常	2020-01-02 14:41:01
☐ settings-eclipse.jar	服务器	正常	2020-01-02 14:42:12

图 7.1-1：文件集管理

在文件集管理页面显示了已添加的文件列表，包含文件集名称，文件来源，文件状态，上传时间。其中文件来源分为“客户端”和“服务器”，文件状态表示文件集对应的文件是否存在，分为“正常”和“文件不存在”。

在文件集管理页面单击“添加文件”后可进入文件上传页面，如下图所示。文件集名称不能重复，默认取文件名称。本机上传不支持修改文件集名称，上传的JAR文件将位于lib/sl目录下。



文件集管理 显示/隐藏详细信息 ⓘ

此页显示了已添加的文件列表(系统内置文件列表consolejars除外)，可以更新、删除所添加文件或者添加新的文件。

文件上传

文件位置 ☒ 本机 ☐ 服务器 选择上传文件位置

文件

描述

文件集的备注信息

图 7.1-2：选择本机上的文件集

从服务器上的添加文件时可修改文件集名称，如果在服务器上选择的文件夹包含class文件，请确保文件路径和包名一致。

说明：

左侧显示为在服务器上启动TongWeb的用户所在的根目录，例如/home/xxx，以及在管理控制台首页**控制台配置**中添加的**服务器选择文件可选目录**，例如下图中的/home/。

注意：

设置服务器选择文件可选目录:首次登录不提供修改功能，用到需要选择文件的组件，页面会给出提示，需要自己手动去单机控制台首页、**设置**服务器选择文件可选目录；目录的设置要求输入绝对路径，

windows不允许设置为系统盘根目录、不允许为系统盘的Windows目录；linux不允许设置为"/"、"/root"、"/etc"。



图 7.1-3：选择服务器上的文件集

单击文件集列表里的文件集名称，可显示资源文件的相关信息。如果是文件夹，则可以查看文件夹下面的所有JAR包或class文件根目录。如下图所示，查看后只能修改描述信息。



图 7.1-4：查看文件集信息

在文件集列表里勾选文件集后单击“**删除**”可删除该文件集。如果该文件集被共享库引用，则无法删

除，应先解除引用关系。如果文件来源是客户端，会同时删除上传的文件。如果文件来源是服务器，则只会删除引用，不会删除服务器源文件。

7.2 共享库

共享库是维护管理应用所需要的类库集合，依赖于文件集，可以配置公有库和私有库。一个文件集可以被多个共享库引用，一个共享库也能被多个应用使用。

展开管理控制台左侧导航树中的“资源”，单击“共享库”，出现如下图所示的“共享库管理”页面。



共享库名称	共享类型	更新时间	创建时间
ant194	私有	2020-01-02 14:23:21	2020-01-02 10:54:58
boot	私有	2020-01-02 14:24:56	2020-01-02 14:24:56
ant197	私有	2020-01-02 14:40:26	2020-01-02 10:54:48

图 7.2-1：共享库管理

在共享库管理页面显示了已添加的共享库列表，包含共享库名称，共享类型，更新时间，创建时间。

在共享库管理页面单击“创建共享库”后进入下图所示的页面。共享库名称不能重复，资源文件必选，左侧为全部文件集，右侧为所选文件集。可以双击列表中的文件来**选择文件**，也可以通过中间的按钮来操作。



返回 创建共享库

* 共享库名称: 共享库名称，唯一标识名

共享类型: 公有 共享库的共享类型

资源文件

未选择资源: 搜索

已选择资源: 搜索

描述: 共享库的描述信息

保存 取消

图 7.2-2：创建共享库

共享库类型分为以下两种：

- 公共共享库：

新建公共共享库后，内存中会创建一个包含此文件列表的ClassLoader。多个应用依赖此公共共享库时，对同一个class只会加载一次。并且应用对共享库中可访问的静态变量修改是相互可见的。

- **私有共享库：**

新建私有共享库后，内存中会创建一个包含此文件列表的ClassLoader。在应用依赖此私有共享库后，会把ClassLoader中包含的jar添加到应用级的ClassLoader中。多个应用同时依赖一个私有共享库时，对同一个class会加载多次。并且共享库中可访问的静态变量是相互独立的。

单击共享库列表里的共享库名称，可查看共享库的相关信息，并可修改共享库类型、资源文件以及描述。如果修改了类型或者资源文件已被应用，则会提示是否确认修改，并且受影响的应用需要重新部署才能使用新的共享库。如果引用的文件集为文件夹，增加或者修改后，则需要修改共享库才能生效。

在共享库列表里勾选共享库后单击“**删除**”可删除该共享库。如果该共享库已被应用引用，则无法删除，应先解除引用关系。

第8章 类加载

Java应用运行时，在class执行和被访问之前，它必须通过类加载器加载使之有效，类加载器是JVM代码的一部分，负责在JVM虚拟机中查找和加载所有的Java类和本地的lib库。

8.1 类加载机制

JVM和TongWeb都提供了多种不同的类加载器，TongWeb中类加载器是一个自上而下的层次结构，最上层是系统的运行环境JVM系统类加载器，最下层是具体的应用类加载器，上下层之间形成父子关系，如下图所示。

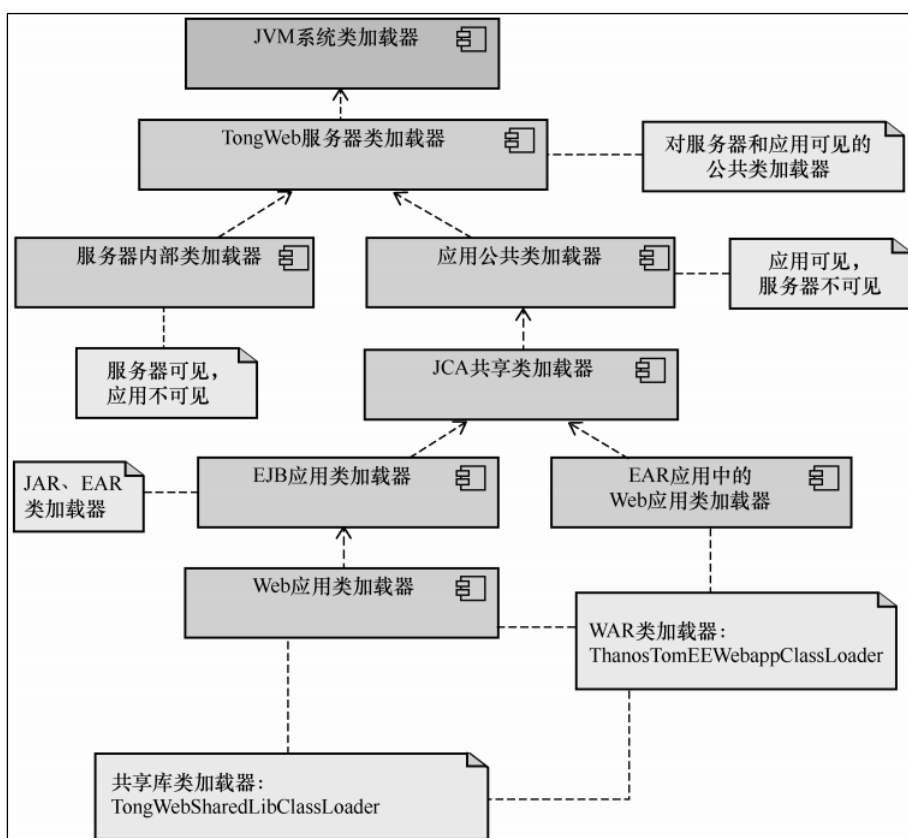


图 8.1-1: 类加载器分层结构图

`${TongWeb_HOME}/conf/tongweb.properties`配置文件中定义了类加载器所需要的类路径，服务器启动时根据此配置文件创建好各个层级的ClassLoader。

- **JVM系统类加载器：**JVM自带的类加载器，分为以下3种。
 - **Bootstrap Classloader：**负责加载核心Java类库。
 - **Launcher\$ExtClassLoader：**负责加载用户需要扩展JVM核心平台时所引用的类，查找的url由java.ext.dirs指定，默认为<JAVA_HOME>/jre/lib/ext。
 - **Launcher\$AppClassLoader：**负责加载服务器启动所需的类，查找的url由-classpath指定。
- **TongWeb类加载器：**加载类的路径对应为`${TongWeb_HOME}/lib`目录，是TongWeb server及应用都可以加载到的公共的类加载器。

- **服务器内部类加载器：**服务器内部Server、Service等核心类的类加载器。
- **应用公共类加载器：**加载类的路径对应为\${TongWeb_HOME}/lib/common及\${TongWeb_HOME}/lib/classes目录，部署的应用可以共享此目录中的类。
- **JCA共享类加载器：**资源适配器共享，对所有应用可见。
- **WEB应用类加载器：**负责加载独立的web模块中的servlets和其他类以及web模块中包含的ejb类。每个web模块会创建一个Web类加载器，用于加载WEB-INF/classes中的类和WEB-INF/lib中的jar包。
- **EJB应用类加载器：**负责加载EAR应用中所包含的公共类、ejb模块和独立ejb应用下所有的类。
- **EAR应用中的WEB应用类加载器：**负责加载EAR应用中所包含的web模块中的servlets和其他类，加载路径是WEB-INF/classes中的类和WEB-INF/lib中的jar包。

注意：

- 每个类加载器负责在自身定义的路径上查找和加载类。
- 一个子类加载器能够委托它的父类加载器查找和加载类，一个加载类的请求会从子类加载器发送到父类加载器，但是从来不会从父类加载器发送到子类加载器。
- 一旦一个类被成功加载，JVM会缓存这个类直至其生命周期结束，并把它和相应的类加载器关联在一起，这意味着不同的类加载器可以加载相同名字的类。
- 如果一个加载的类依赖于另一个或一些类，那么这些被依赖的类必须存在于这个类的类加载器查找路径上或者父类加载器查找路径上。
- 如果一个类加载器以及它所有的父类加载器都无法找到所需的类，系统就会抛出ClassNotFoundException异常或者NoClassDefFoundError错误。

8.2 类加载模式

TongWeb提供父优先和子优先两种类加载模式，该模式决定类加载器在查找一个类的时候，是优先查找类加载器自身指定的类路径还是优先查找父类加载器上的类路径。

Web应用和EJB应用支持两种类加载模式，tongweb-web.xml中的delegate属性用于决定应用使用子优先模式还是父优先模式，delegate默认使用子优先模式。另外在应用部署时，可以选择父优先还是子优先模式，管理控制台上设置的加载模式优先于tongweb-web.xml中delegate配置。tongweb-web.xml中delegate属性配置优先于其他文件中的delegate配置。当delegate设置为true时是父优先，设置为false时是子优先。具体的运用代码如下。

```
<tongweb-web-app>
<loader delegate="true" />
</tongweb-web-app>
```

各Classloader默认使用父优先的类加载模式，即当父类加载器找不到所加载的资源时，才使用当前的子类加载器。

8.2.1 父优先

在加载类的时候，在从类加载器自身的类路径上查找加载类之前，首先尝试在父类加载器的类路径上

查找和加载类。

当ClassLoader被请求加载某个类时，它首先委托自己的父类加载器去加载，若父类加载器能加载，则返回这个类所对应的类对象；如果父类加载器不能加载，才由当前类加载器去加载；如果都不能成功，则系统就会抛出ClassNotFoundException异常或者NoClassDefFoundError错误。

例如，在Web应用(.war)中lib下的utility.jar中存在类A.class，\${TongWeb_HOME}/lib目录下的package.jar中也存在一个同名的类A.class。若采用父优先的类加载策略，则当应用级的ClassLoader被请求加载类A.class时，返回的是其父类加载器加载的package.jar中的A.class对象，而不是utility.jar中的类A.class。

8.2.2 子优先

在加载类的时候，首先尝试从自己的类路径上查找加载类，在找不到的情况下，再尝试父类加载器类路径。

当ClassLoader被请求加载某个类时，它首先试图自己去加载，如果能加载成功，则返回这个类所对应的Class对象；如果不能加载成功，才由该类加载器的父类加载器去加载；如果都不能成功，则系统就会抛出ClassNotFoundException异常或者NoClassDefFoundError错误。

例如，对于前面类加载的示例，使用子优先的类加载策略，则最终被加载的类为utility.jar中的A.class，这个对象由Application ClassLoader加载。

8.3 类加载推荐策略

在组装企业应用时，唯一性是处理类加载问题的一个很好的指导原则，即相同的类只放置在一个地方，从一种途径加载，TongWeb推荐的策略如下。

1. EJB应用自身依赖的类，封装在自身的EJB和JAR中。
2. Web应用自身依赖的类，或者以JAR包的形式放置于WAR中WEB-INF/lib目录下，或者以类的形式放置于WEB-INF/classes目录下。
3. 在一个EAR应用中，多个EJB或Web模块共同使用的公用包，直接以普通JAR包方式放置于EAR根目录下。在一个EAR应用中，多个Web模块中包含类名相同，功能（代码）不同的类，使用子优先模式。避免EAR的类加载器选择Web模块中的某一个类（取决于Web模块在部署文件中的顺序）共享给所有Web模块使用。
4. 在部署多个应用，这些应用要共享相同类且不需要更多服务配置时，可以选择以下三种模式：
 - a) 一般共享模式

将需要共享的类放在 TongWeb 默认提供的类共享目录 lib/common 和 lib/classes 目录下，其中 lib/common 目录下放 JAR 包，lib/classes 目录下放 class。启动服务器后会把这两个目录的文件加载并创建共享的 ClassLoader。
 - b) 公共共享库模式

在TongWeb资源文件集中选取需要作为公共共享库的JAR包或文件夹资源，服务器会立即创建包含以上jar包的ClassLoader，应用部署时关联上公共共享库即可。
 - c) 私有共享库模式

在TongWeb资源文件集中选取需要作为私有共享库的JAR文件或文件夹资源，服务器会在有应

用关联到此私有共享库并部署时创建包含以上jar包的ClassLoader。

注意：a)、b)中的共享模式是所有应用功能可以实现多个应用共享类，但所对应的 class 只会被加载一次，并且应用对共享库中可访问的静态变量的修改是相互可见的，所以可能会出现不同应用相互覆盖等冲突问题，如果想实现每个应用使用的共享类ClassLoader相互独立，则应采用c)模式。

5. 如果部署的应用希望依赖非服务器默认提供的javaee api版本，如annotation -api，可以选择将期望版本的jar包放置在lib/endorsed中实现api替换。此功能是基于Java提供的endorsed技术，endorsed可以简单理解为-Djava.endorsed.dirs指定的目录里放置的jar文件，将有覆盖系统API的功能。但是能够覆盖的类是有限制的，其中不包括java.lang包中的类（出于安全的考虑）。

说明：

使用此参数是因为java是采用双亲委派机制进行加载class类的。而jdk提供的类只能由类加载器Bootstrap进行加载。如果要在应用中替换掉jdk中的某个类是无法做到的，所以java提供了endorsed来替换系统中的类。

6. 类加载策略的配置：

Web应用

配置tongweb-web.xml文件：

```
<?xml version="1.0" encoding="UTF-8"?>
<tongweb-web-app>
  <prefer-app-packages>
    <package-name>com.example2.*</package-name>
    <class-name>com.example.demo.XssFilter</class-name>
  </prefer-app-packages>
  <prefer-app-resources>
    <resource-name>application.properties</resource-name>
  </prefer-app-resources>
</tongweb-web-app>
```

- prefer-app-packages
可以指定包名、类名、资源文件，配置后，始终由应用加载，若应用类加载器加载不到则抛出异常。
- prefer-app-resources
可以指定资源文件，配置后，只在应用类加载器中搜索。

EAR应用

配置tongweb-ejb-jar文件：

```
<?xml version="1.0" encoding="UTF-8"?>
<tongweb-ejb-jar>
  <prefer-app-packages>
    <package-name>org.example2.demo.*</package-name>
    <class-name>org.example.demo.ejbtest2.service.MemberRegistrationBean</class-name>
  </prefer-app-packages>
  <prefer-app-resources>
    <resource-name>META-INF/faces-config.xml</resource-name>
  </prefer-app-resources>
</tongweb-ejb-jar>
```

1. prefer-app-packages、prefer-app-resources配置说明同Web中相同。
2. EAR中包含多个ejb应用时，会合并配置，包含的war包按Web应用的逻辑。

注：

扫描注解时，容器类加载器会扫描到EAR中WAR包内的类，会加载进来，但是实际使用的还是web应用类加载器加载的CLASS实例（可以将war包中的类配置到tongweb-ejb-jar.xml文件中，即可避免）。

7. 同一目录下jar文件加载顺序的配置：

Web应用

配置WEB-INF/tongweb-web.xml文件,应用WEB-INF/lib下jar包顺序：

```
<?xml version="1.0" encoding="UTF-8"?>
<tongweb-web-app>
  <property name="mappingFromAsRole" value="true"/>
  <property name="cookie-samesite" value="Lax"></property>
  <jar-orders>
    <jar-order jar-name="tongweb-deploy-agent.jar" />
    <jar-order jar-name="tongweb-deploy-client.jar" />
    <jar-order jar-name="ddddd.jar" />
  </jar-orders>
</tongweb-web-app>
```

EAR应用

配置META-INF/application.xml文件：

```
<?xml version="1.0" encoding="UTF-8"?>
<application
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://java.sun.com/xml/ns/javaee"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
  http://java.sun.com/xml/ns/javaee/application_5.xsd" id="Application_ID" version="5">
  <display-name>libjarTest</display-name>
  <module id="esb-client-api-1.1">
```

```

    <java>esb-client-api-1.1.jar</java>
    <java>HelloWorldJar.jar</java>
    <java>CelloWorldJar.jar</java>
  </module>
  <module id="HelloWorldJar">
    <java>HelloWorldJar.jar</java>
  </module>
  <module id="CelloWorldJar">
    <java>CelloWorldJar.jar</java>
  </module>
</application>

```

ear应用如果配置application.xml文件,所有ejb、web模块都得定义;只对java标签定义的文件排序。

8.4 类加载参数

在服务器启动和应用部署过程中,可供参考的参数如下。

- **exclusions.list:** 位置为\${TongWeb_HOME}/bin/conf,默认无。
- **default.exclusion:** 位置为\${TongWeb_HOME}/lib/tongweb.jar,默认有。
- **tongweb.properties:** 位置为\${TongWeb_HOME}/conf,默认有。

说明:

- 父优先: lib, **commons**公有库, ear/APP-INF私有库
- 子优先: ear/APP-INF私有库, lib, **commons**公有库

生效范围:

应用服务器会取exclusions.list中的内容作为启动过程中JAR包等的过滤列表,如果该列表不存在,则取默认列表(即default.exclusion)作为基本的过滤列表。在启动过程中,还会读取TongWeb.properties,取其TongWeb.util.scan.StandardJarScanFilter.jarsToSkip的值,加入到过滤列表中,在随后的加载过程中,会将环境变量中的TongWeb.util.scan.StandardJarScanFilter.jarsToSkip设置为上述列表值。

说明:

上述参数描述了应用服务器在启动和应用部署过程中过滤掉的类和JAR包的列表。

附录A 应用配置文件

在应用中可以配合Java EE应用中标标准描述文件web.xml和ejb-jar.xml等文件，通过配置TongWeb的自定义部署描述文件，进一步地自定义应用。在Web应用中可以配置web.xml和tongweb-web.xml（如图1.1-1 Web应用的结构）。在含有EJB的应用中可以配置ejb-jar.xml、tongweb-ejb-jar.xml和persistence.xml（如图1.1-3 EJB应用的结构）。

A.1 default-web.xml

`${TongWeb_HOME}/conf/default-web.xml`是系统级公共基础配置，其定义的相关参数如下。

表 A.1-1: default-web.xml相关参数说明

参数名	参数类型	默认值	说明
development	Boolean	true	Jasper 是否在开发模式中使用
fork	Boolean	true	Ant 是否应该将其 java 编译的 JSP 页面分开
keepGenerated	Boolean	true	是否要保留生成的 Java 文件
trimSpaces	Boolean	false	指令或操作之间的空格是否应该修剪
isPoolingEnabled	Boolean	true	是否启用标记处理程序池
mappedFile	Boolean	true	是否要支持“映射”文件？这将产生在 JSP 文件的每一行有一个 print 语句的 servlet，是一个非常好的调试功能。
classDebugInfo	Boolean	true	是否要在类文件中包含调试信息
checkInterval	Int	0	后台编译线程检查间隔（秒）
isSmapSuppressed	Boolean	false	JSR45 调试的 SMAP 信息是否被禁止生成
isSmapDumped	Boolean	false	JSR45 调试的 SMAP 信息是否转储到文件中
genStringAsCharArray	Boolean	false	文本字符串是否作为字符数组生成
scratchDir	File		生成的 servlet 文件目录
ieClassId	String		IE 的版本 4 和 5 需要有这个。如果它在未来发生变化，所需的只是要有 ieClassId=“<value>类型的 jsp initParam 默认值： clsid:8AD9C840-044E-11D1-B3E9-00805F499D93
classpath	String	null	编译生成 servlet 时使用的类路径
compiler	String	null	要使用的编译器
compilerTargetVM	String	1.7	编译器目标 VM
compilerSourceVM	String	1.7	编译器源 VM

参数名	参数类型	默认值	说明
compilerClassName	String	null	编译器类名
TldCache	TldCache	null	TLD uri、资源路径和已解析文件的缓存
jspConfig	JspConfig	null	JSP 配置信息
javaEncoding	String	UTF8	生成 JSP 的 Java 平台编码
modificationTestInterval	int	4	修改测试间隔
recompileOnFail	Boolean	false	失败后是否立即尝试重新编译
xpoweredBy	Boolean		X-Powered-By 响应头的生成是否已启用/禁用
displaySourceFragment	Boolean	true	是否在异常消息中包含一个源片段
maxLoadedJsps	int	-1	每个 web 应用程序加载的 JSP 的最大数目。如果还有更多加载的 JSP，它们将被卸载。
jspIdleTimeout	int	-1	卸载 JSP 之后的空闲时间（秒），如果未设置或小于或等于 0，则不卸载任何 JSP。
strictQuoteEscaping	Boolean	true	JSP.1.6 是否严格应用于使用 scriptlet 定义的属性
quoteAttributeEL	Boolean	true	当 EL 用于 JSP 属性值时，是否引用 JSP.1.6 中描述的属性是否应用于表达式

A.2 tongweb-web.xml

A.2.1 属性配置顺序

tongweb-web.xml的元素配置，需要严格按照xsd定义的顺序来配置，配置顺序如下所示。

1. security-role-mapping
2. resource-links
3. manager
4. loader
5. use-myfaces
6. listeners
7. watched-resource
8. jar-scanner
9. property
10. context-root
11. session
12. sharedlib-config
13. prefer-app-packages
14. prefer-app-resources

15. jar-orders

A. 2. 2 属性配置说明

表 A. 2. 2-1: tongweb-web.xml 相关属性说明

元素/属性名	说明
context-root	Web 应用访问前缀，该值可能被部署 Web 应用时的"应用前缀"属性覆盖。 自动部署配置 context-root 属性时，需要添加"/"，在控制台才能访问。 例如： <context-root>/</context-root> 或者 <context-root>/test</context-root>
元素/属性名	说明
security-role-mapping	把角色映射到当前安全域中的用户或组。最多配置一个
role-name	被映射的角色名称，根据用户名或者用户所在组进行映射。 角色名(唯一)，对应 JavaEE 标准部署描述符文件中的 security-role 元素
principal-name	拥有该角色的用户名，如果有 group-name，用户名可以写 0 个或者多个，如果没有 group-name，至少要有 1 个用户名 注意：在没有 group-name 时，只有 principal-name 设置的用户名可以通过安全认证，需要配置多个用户名时，用冒号分隔
group-name	拥有该角色的组名，如果有 principal-name，组名可以写 0 个或者多个，如果没有 principal-name，至少要有 1 个组名 注意：当应用绑定的安全域的分配组和应用中的 group-name 设置的组相同，则应用绑定的安全域中的所有用户都可以通过安全认证。需要配置多个组名时用冒号分隔
元素/属性名	说明
resource-links	用于指定对外部资源引用的声明，最多配置一个，包含 0 个或多个 resource-link 元素
name	被创建的资源连接的名称，相对于 java:comp/env 上下文
global	在全局的 jndi 上下文中，连接的全局资源的名称
type	资源类型，全类名。如：java.lang.Integer
元素/属性名	说明
loader	配置 Web 应用用来加载资源的类加载方式，最多配置一个
delegate	当为 true 时，加载类时采用的是双亲委托机制，即先从父类加载器加载类，依次递归，如果父类加载器可以完成类加载任务，就成功返回；只有父类加载器无法完成

元素/属性名	说明
	此加载任务时,才由加载 web 应用的类加载器从 web 应用中查找和加载类。为 false 时, 反之。默认 false
search-external-first	当 true 时, 首先从 external-classpath 配置的路径寻找资源。false 时, 从 classload 加载。默认 false
external-classpath	该属性可以配置目录或者 jar 的路径。目录时,首先在该目录下寻找 class 进行加载; 为 jar 时, 将首先在该 jar 中寻找 class 进行加载, 同时支持*.jar 格式, 即首先在上 级目录下的所有 jar 中寻找; 目录或者 jar 不存在时自动跳过, 多个路径使用分号 分隔。
元素/属性名	说明
watched-resource	监视 web 应用的静态资源, 如果被监视的资源有更新, 则重部署应用。 可以包含一个或多个 property 元素。以下的 name 和 value 为 property 的属性, 例 如: <property name="web" value="WEB-INF/web.xml">
name	任意有意义的名称
Value	要监视的资源的物理路径
元素/属性名	说明
listeners	该 listener 用于监听 Context 组件(即当前应用)的事件。通过这个 listener, 可以实现 在监听到 Context 的事件(例如应用启动前后, 应用卸载等)发生时执行用户自定义 的逻辑。可以包含 0 个或多个 listener 元素。 以下的 name 和 class-name 为 listener 的属性, 例如: <listener name="test" class-name="com.tongweb.web.monitor. WebAppMonLifecycleListener">
name	任意有意义的名称
Value	为一个实现了 com.tongweb.web.thor.LifecycleListener 接口的类。
元素/属性名	说明
jar-scanner	它用于扫描 web 应用的 jar 文件
scan-all-directories	默认 false。如果为 true, 到类加载路径下的所有目录去判断其是否是展开的 jar 文 件
scan-all-files	默认 false。如果是 true, 检查所有的在类加载路径下的文件是不是 jar 文件, 而不 仅仅是.jar 结尾的文件
scan-class-path	默认 false。如果为 true, 除了 web 应用, 所有的其他的类加载路径都会被扫描, 以加载 jar 文件
元素/属性名	说明

元素/属性名	说明
manager	设置 session 管理器的实现类名，session 管理器是 web 容器用来创建、维持 Web 应用的 HTTP Session 的组件，最多可以设置一个
max-active-sessions	最大活跃 session 数，-1（默认）表示无限制
max-inactive-interval	会话超时时间，默认为 30 分钟
session-id-length	session id 的长度，默认为 16
元素/属性名	说明
session	应用中 Session 相关的属性设置
cookie-properties	设置 Session Cookie 相关属性，包括 Session Cookie 的名字，如果不设置这个属性，将使用默认值：JSESSIONID。
元素/属性名	说明
prefer-app-packages	可以指定包名、类名、资源文件。配置后，始终由应用加载，若应用类加载器加载不到则抛出异常。
元素/属性名	说明
prefer-app-resources	可以指定资源文件，配置后，只在应用类加载器中搜索。
元素/属性名	说明
use-myfaces	已弃用，若出现，则需要严格按照 xsd 定义的顺序配置。

表 A.2-2: session 高可用相关属性说明

元素/属性名	说明
property	web 应用其他可配的属性
以下为 session 高可用相关属性配置	
tongdatagrid-group-name	集群组名称，默认为 dev
tongdatagrid-group-password	集群组密码，默认值为 dev-pass
tongdatagrid-cluster-members	要连接到的集群节点 IP。可以指定 1 个或多个，用逗号分隔；可以加端口号也可以不加。客户端按此顺序尝试连接服务端，直到连接上。
tongdatagrid-asyncwrite	是否异步写入，默认 false（同步）
tongdatagrid-timeout	对缓存集群的操作超时, 单位: 毫秒, 默认值 100

元素/属性名	说明
tongdatagrid-stick	Web 集群亲和设置开关，默认 true（亲和）。设置为 false 表示非亲和。该参数与参数-Dwebcluster.session.sticky 至少有一个设置了非亲和，则 TongWeb 集群将运行在非亲和模式；只有在两个参数都是亲和配置（默认值）下，TongWeb 集群才会运行在亲和模式下。
tongdatagrid-enabled	是否启用集群功能，默认值为 false
multicast-group	组播分组的 IP 地址。当要创建同一个网段的集群时，需要配置这个参数。取值范围从 224. 0. 0. 0 到 239. 255. 255. 255，默认 224. 2. 2. 3
multicast-port	组播协议启用套接字的端口（socket port），这个端口用于 TongDataGrid 监听外部发送来的组网请求。默认 54327
multicast-time-to-live	组播协议发送包的生存时间周期（TTL）
multicast-timeout-seconds	当节点启动后，这个参数（单位：秒）指定了当前节点等待其他节点响应的的时间周期。例如，设置为 60 秒时，每一个节点启动后通过组播协议广播消息。如果主节点在 60 秒内返回响应消息，则新启动的节点加入这个主节点所在的集群。如果设定时间内没有返回消息，那么节点会把自己设置为一个主节点，并创建新的集群（主节点可以理解为集群的第一个节点）。默认值为 2 秒
trusted-interfaces	可信任成员的 IP 地址。当一个节点试图加入集群，如果其不是一个可信任节点，他的加入请求将被拒绝。可以在 IP 的最后一个数字上使用通配符（*）来设置一个 IP 范围（例如：192. 168. 1. * 或 192. 168. 1. 100-110）
interfaces	指定 TongDataGrid 使用的网络接口地址，服务器可能存在多个网络接口，因此需要限定可用的 IP 地址。如果配置的 IP 地址找不到，会输出一个异常信息，并停止启动节点。

表 A. 2-3：共享库依赖配置说明

元素/属性名	说明
property	web 应用其他可配的属性
以下为共享库依赖配置。	
sharedlib-config	共享库配置的顶级元素，最多可以设置一个。例如： <pre><sharedlib-config> <shared-lib-relation/> <app-exclude/> </sharedlib-config></pre>
shared-lib-relation	关联共享库的 Group，最多可以设置一个。例如： <pre><shared-lib-relation> <shared-lib-ref name="springlibs1"/> <shared-lib-ref name="springlibs2"/></pre>

元素/属性名	说明
	</shared-lib-relation>
shared-lib-ref	使用 name 指定已配置好的共享库名称，需要先在管理控制台配置。此标签可多个。如果当前 name 指定的共享库不存在，则忽略此条配置。例如： <pre><shared-lib-ref name="springlibs1"> <exclude-jar> <path>/home/userlib/service.jar</path> </exclude-jar> </shared-lib-ref></pre> 注：path 为绝对路径，可以查看\${TongWeb_HOME}/conf/assets.xml 里面路径。
exclude-jar	配置当前关联的共享库中要排除的 jar 包。子标签 path 指定包的绝对位置。可为空
app-exclude	配置排除当前应用自带的 jar 包，最多可以设置一个。即排除 lib/xx.jar。子标签 path 指定包的全名称。可为空 例如： <pre><app-exclude> <path>aopalliance-1.0.jar</path> <path>commons-digester-2.1.jar</path> </app-exclude></pre>

A.3 ejb-jar.xml

表 A.3-1: ejb-jar.xml 相关属性说明

元素/属性名	说明
ejb-jar	EJB 部署描述符的根元素。它包含 ejb-jar 文件的可选描述、可选显示名称、可选小图标文件名、可选大图标文件名、所有包含的企业 bean 的必需结构信息、可选应用程序集描述符以及 ejb jar 的 ejb 客户端 jar 文件的可选名称。
description	ejb-jar文件的描述性说明
display-name	ejb-jar文件显示的名称
small-icon	小图标(16 x 16)的名字，这个名称是在ejb-jar文件的相对路径。文件必须是jpg或者是gif格式，文件名必须以.jpg.或者gif为后缀。图标可以被工具使用
large-icon	大图标(32 x 32)的名字，这个名称是在ejb-jar文件的相对路径。文件必须是jpg或者是gif格式，文件名必须以.jpg.或者gif为后缀。图标可以被工具使用
enterprise-beans	该元素包含一个或多个企业 bean 的声明
元素/属性名	说明
session	该元素声明一个会话bean
description	简短的描述
display-name	会话bean通过工具将显示的名称

元素/属性名	说明
small-icon	小图标(16 x 16)的名字，这个名称是在ejb-jar文件的相对路径。文件必须是jpg或者是gif格式，文件名必须以.jpg.或者gif为后缀。图标可以被工具使用
large-icon	大图标(32 x 32)的名字，这个名称是在ejb-jar文件的相对路径。文件必须是jpg或者是gif格式，文件名必须以.jpg.或者gif为后缀。图标可以被工具使用
ejb-name	企业bean的名称，由ejb-jar文件的编写者命名，这个名称在同一个ejb-jar文件中必须是唯一的。企业bean的代码不依赖于它，并与它隔离。所以企业bean的名称可以在应用装配的过程中改变，而不会影响企业bean的功能。在发布描述器中ejb-name与应用发布者即将分配给企业bean的home接口的JNDI的名称，没有结构性的关系。名称必须定义的有意义。
home	企业bean的具有完全限定名(具有完整的包名)的home接口
remote	企业bean的具有完全限定名(具有完整的包名)的remote接口
ejb-class	企业bean实现类的完全限定名(具有完整的包名)
session-type	描述会话bean是否有状态的会话，或者无状态会话。会话型元素必须是Stateful/Stateless其中之一。
transaction-type	制定是事务类型，必须是Bean/Container其中之一
transaction-scope	指定企业bean是否要求在其方法中必须使用分布式事务，以确定是否可以使用本地事务优化。必须是Local/Distributed其中之一。该元素是可选的，如果未指定，则容器必须使用分布式事务。
元素/属性名	说明
env-entry	企业bean的环境条目的声明。该声明由可选的描述，环境条目的名称，以及可选值组成
description	简短的描述
env-entry-name	企业bean的环境条目名称
env-entry-type	具有完全限制的企业bean代码期望得到的Java类型值。合法值： java.lang.Boolean, java.lang.String, java.lang.Integer, java.lang.Double, java.lang.Byte, java.lang.Short, java.lang.Long, java.lang.Float
env-entry-value	企业bean的环境条目的值
元素/属性名	说明
ejb-ref	用于引用到另一个企业bean的home声明。该声明是一个可选描述：在引用的企业bean代码中使用EJB引用名称，被引用企业bean的预期类型，被引用企业bean的预期home和remote接口，以及一个可选的ejb-link信息。可选的 ejb-link元素用来指定引用的企业bean，它通常用在包含组装应用程序的ejb-jar.xml文件。
description	简短的描述
ejb-ref-name	EJB引用的名称。EJB参照是企业bean环境中的一条条目。建议名称的前缀是“EJB/”

元素/属性名	说明
ejb-ref-type	被引用的企业bean的类型。必须是Entity/Session其中之一
home	企业bean的home接口的完全限定名称
remote	企业bean的远程接口的完全限定名称
ejb-link	指定EJB引用链接到在ejb-jar文件中的另一个企业Bean。该元素的值必须是在同一个ejb-jar文件中的企业bean，或者在同一个应用程序单元的另一个ejb-jar文件中 ejb-name定义的企业bean。
元素/属性名	说明
security-role-ref	在企业bean代码中安全角色引用的声明。该声明包含一个可选的描述，在代码中使用的安全角色名称，并定义安全角色的可选链接。该元素的值必须是作为参数传递给EJBContext.isCallerInRole（字符串角色名）方法的字符串。role-link元素的值必须是在该元素中定义的安全角色之一的名称
description	简短的描述
role-name	安全角色的名称
role-link	已经定义的安全角色的连接。role-link元素的值必须是在security-role元素中定义过的
元素/属性名	说明
security-identity	指定调用者的安全标识是否用于输入调用bean的方法执行，或者是否使用特定的run-as-identity
description	简短的描述
use-caller-identity	指定调用者的安全标识，可以用作对企业bean的方法执行的安全标识
元素/属性名	说明
run-as-specified-identity	指定run-as-identity 将用于企业 bean 的方法的执行
description	简短的描述
role-name	安全角色的名称
元素/属性名	说明
resource-ref	企业bean对外部资源的引用的声明。它由可选的描述、资源工厂引用名称、企业bean代码所期望的资源工厂类型的指示和认证类型(bean或者container)组成。
description	简短的描述
res-ref-name	资源工厂引用的名称
res-type	数据源的类型。类型由数据源期望实现的Java接口(或类)指定
res-auth	定义是由企业bean的代码作为资源管理器，还是由容器作为资源管理器。发布者可以对容器提供支持，这个元素的值必须是Application和Container中的一个。
元素/属性名	说明
resource-env-ref	企业bean对与企业bean环境中的资源相关联的受管理对象的引用的声明。它由

元素/属性名	说明
	可选的描述、资源环境引用名称和企业bean代码期望的资源环境引用类型的指示组成。用于实体、消息驱动和会话。示例：jms/StockQueue javax.jms.Queue
description	简短的描述
resource-env-ref-name	资源环境引用的名称。值是 Enterprise bean 代码中使用的环境条目名称
resource-env-ref-type	资源环境引用的类型
元素/属性名	说明
entity	描述 entity bean
description	简短的描述
display-name	通过工具将显示和被引用的名称
small-icon	小图标(16 x 16)的名字，这个名称是在 ejb-jar 文件的相对路径。文件必须是 jpg 或者是 gif 格式，文件名必须以 .jpg 或者 gif 为后缀。图标可以被工具使用
large-icon	大图标(32 x 32)的名字，这个名称是在 ejb-jar 文件的相对路径。文件必须是 jpg 或者是 gif 格式，文件名必须以 .jpg 或者 gif 为后缀。图标可以被工具使用
home	Enterprise bean 的具有完全限定名(具有完整的包名)的 home 接口
remote	Enterprise bean 的具有完全限定名(具有完整的包名)的 remote 接口
ejb-class	Enterprise bean 实现类的完全限定名(具有完整的包名)
persistence-type	指定 entity bean 的持久性管理的类型,必须是 Bean / Container 其中之一
prim-key-class	指定具有完全限定的 entity bean 的主键类的名称。如果主键类的定义依赖发布时间, 主键类元素的值应该是 java.lang.Object
reentrant	指定一个实体 bean 是否可重入。该元素必须是 True 或 False
CMP-version	指定容器管理持久性的实体 bean 的版本。合法值为 1.x 和 2.x，默认值是 2.x。
abstract-schema-name	指定与 cmp-version 2.x 实体 bean 对应的抽象模式名（表名），用在 EJB QL 查询。例如，实体 bean 类是 com.acme.commerce.OrderBean，命名为 OrderBeanTable
元素/属性名	说明
cmp-field	描述容器管理的字段，包括可选的字段描述和字段的名称
description	简短的描述
field-name	指定容器管理的字段的名。名称必须是 Enterprise bean 的公共字段或者是它的超类中的公共字段。
primkey-field	指定容器管理的 Bean 的主键字段。必须是在 cmp-field 元素中声明过的，类型必须和主键类一样。如果主键映射为多个容器管理的字段(也就是说是一个复合主键)，则没必要使用这个元素。在这种情况下，主键类的字段必须是公共的，它们的名称必须与包含主键的 entity bean 的字段名相符合
元素/属性名	说明
query	指定查找器或选择查询。包含查询的可选描述、使用 finder 或 select 方法的规范，以及 EJB QL 查询字符串或查询规范
description	简短的描述

元素/属性名	说明
query-spec	指定finder或select查询，包含查询语义的精确描述。query-spec元素只用于指定那些在EJB QL中无法捕获语义的查询。
ejb-ql	定义finder或select的EJB QL查找器或选择查询字符串。此元素在查询元素的范围内定义，指定使用查询的查找程序或select方法，指定查询的实体bean的有效EJB QL查询字符串。如果没有指定ejb-ql元素，则必须使用query-spec元素来指定查询的语义
query-method	为finder或select查询指定方法
method-name	包含企业bean的方法或使用星号（*）字符的名称。使用星号时表示企业bean的remote和home接口中的所有方法
元素/属性名	说明
method-params	包含方法参数的完全限定的Java类名的列表
method-param	方法参数的完全限定的Java类名
元素/属性名	说明
message-driven	声明了一个消息驱动bean
description	简短的描述
display-name	通过工具将显示和被引用的名称
small-icon	小图标(16 x 16)的名字，这个名称是在ejb-jar文件的相对路径。文件必须是jpg或者是gif格式，文件名必须以.jpg.或者gif为后缀。图标可以被工具使用
large-icon	大图标(32 x 32)的名字，这个名称是在ejb-jar文件的相对路径。文件必须是jpg或者是gif格式，文件名必须以.jpg.或者gif为后缀。图标可以被工具使用
home	企业bean的具有完全限定名(具有完整的包名)的home接口
remote	企业bean的具有完全限定名(具有完整的包名)的remote接口
ejb-class	企业bean实现类的完全限定名(具有完整的包名)
transaction-type	制定是事务类型，必须是Bean/Container其中之一
transaction-scope	指定企业bean是否要求在其方法中必须使用分布式事务，以确定是否可以使用本地事务优化。必须是Local/Distributed其中之一。该元素是可选的，如果未指定，则容器必须使用分布式事务。
jms-message-selector	指定jms消息选择器，用于确定消息驱动bean将接收哪些消息
jms-acknowledge-mode	指定jms AUTO_ACKNOWLEDGE或DUPS_OK_ACKNOWLEDGE消息确认语义,应用于使用bean管理的事务划分的消息驱动bean的onMessage消息。必须auto-acknowledge/dups-ok-acknowledge之一
message-driven-destination	消息驱动的目的地元,向部署人员提供关于消息驱动bean是用于队列还是主题的建议。声明包括message驱动bean的预期目的地的类型。如果消息驱动bean被分配给主题，则应使用持久订阅还是非持久订阅的可选声明。
jms-destination-type	jms目的地的类型，例如： javax.jms.Queue

A.4 tongweb-ejb-jar.xml

tongweb-ejb-jar.xml 是一个 EJB 的自定义配置文件，可以定义多个 **ejb-deployment** 节点。每个 ejb-deployment 节点的内容都代表着一个 EJB，可以在这个自定义部署描述文件“定制”每一个 EJB 的内容，代码如下。

```
<tongweb-ejb-jar>
  <ejb-deployment ejb-name =""> //ejb的名称
    <jndi></jndi> //Jndi全局资源
    <resource-link res-id="" res-ref-name=""></resource-link> //资源映射
    <pool></pool> //ejb容器定制映射
  </ejb-deployment>
</tongweb-ejb-jar>
```

tongweb-ejb-jar.xml 定义的具体属性描述如下。

表 A.4-1: ejb-deployment 节点属性说明

元素/属性名	说明
ejb-name	ejb 名称，对应标准部署描述符文件（ejb-jar.xml）中的 ejb-name 元素
resource-link	定义资源引用
pool	配置该无状态会话 bean 或消息驱动 bean 的实例池属性

表 A.4-2: Jndi 属性说明

元素/属性名	说明
name	定义 ejb 的接口的全局 jndi 名
interface	ejb2 中设置 home 接口名，ejb3 中设置 local 或者 remote 接口名。无接口的 ejb 直接使用 ejb 类名

表 A.4-3: resource-link 属性说明

元素/属性名	说明
res-id	引用的自定义 jndi 名
res-ref-name	引用的全局 jndi 名

表 A.4-4: pool 属性说明

元素/属性名	说明
max	池里面最大实例数

元素/属性名	说明
strict	是否禁止池溢出，设置为 true 超过最大实例数，不再分配
min	池里面最小实例数
garbage-collection	是否支持垃圾回收软引用，设置为 true，池里面的实例都是软引用
max-age	实例存在最大生存时间，单位为小时，默认为 0，永不超时
replace-aged	超过实例最大生存时间，再进行实例替换
replace-flushed	当进行池刷新时，进行实例替换
max-age-offset	延迟参数
idle-timeout	空闲超时时间，单位为分钟，默认为 0，永不超时
interval	池周期扫描时间，默认为 5 分钟

A.5 persistence.xml

表 A.5-1: persistence.xml 相关属性说明

元素/属性名	说明
persistence	持久化配置的根元素
persistence-unit	数据持久化单元，包含name属性用于定义持久化单元的名字（name必选，空值也合法）；transaction-type指定事务类型（可选）
description	简短的描述
provider	javax.persistence.PersistenceProvider接口的一个实现类（可选）
jta-data-source	持久化提供商使用的JTA数据源的全局JNDI名称（可选）
non-jta-data-source	持久化提供商使用的non-JTA数据源的全局JNDI名称（可选）
mapping-file	声明orm.xml所在位置（可选）
jar-file	以包含persistence.xml的jar文件为基准的相对路径，添加额外的jar文件（可选）
class	显式列出实体类，在Java SE 环境中应该显式列出（可选）
exclude-unlisted-classes	声明是否扫描jar文件中标注了@Entity的类加入到上下文（可选）。如果不扫描，设置成true。
properties	厂商专有属性列表（可选）
property	厂商专有属性（可选），包含name和value属性。

附录B 术语及缩略语

名称	说明
TongWeb(简称TW)	东方通应用服务器
\${TongWeb_HOME}	TongWeb的安装路径
EJB	Enterprise JavaBean, 基于分布式事务处理的企业级应用
JSP	Java Server Pages, Java服务器端页面
JVM	Java Virtual Machine, Java虚拟机
AJP	Apache JServ Protocol, 定向包协议
JDBC	Java Database Connectivity, Java数据库连接
SNMP	简单网络管理协议
TongWeb域	逻辑服务器管理

